

Deep Learning 2

EECS 492

Introduction to Artificial Intelligence

Reminders

- HW1 due 10/6
- Optional exam practice and review 10/8
 - Sign up here:
<https://forms.gle/pzz6M81ASLBbVAVK9>

Self-Attention Networks: Transformers

The Neural Architecture Powering Modern AI

- Transformers currently the preferred neural network architecture.
- They power current chat bots like ChatGPT, Bard, Llama, Claude, etc.
- For example: GPT stands for Generative Pre-trained **Transformer**.

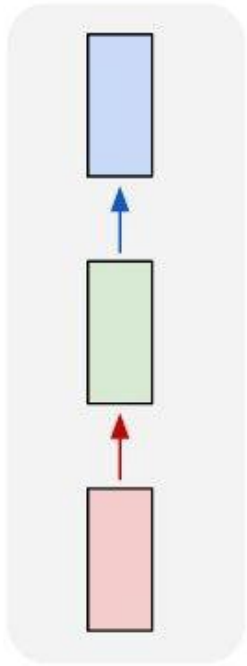


Transformers

- Maps **sequences** of input vectors to sequences of output vectors of the same length
- Made up of stacks of **transformer blocks**, which are combinations of:
 - Feedforward networks
 - **Self-attention** layers (**key innovation**)

“Vanilla” Neural Network

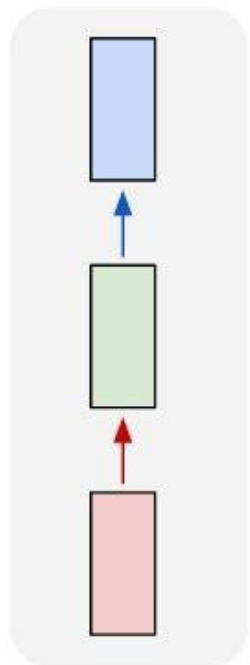
one to one



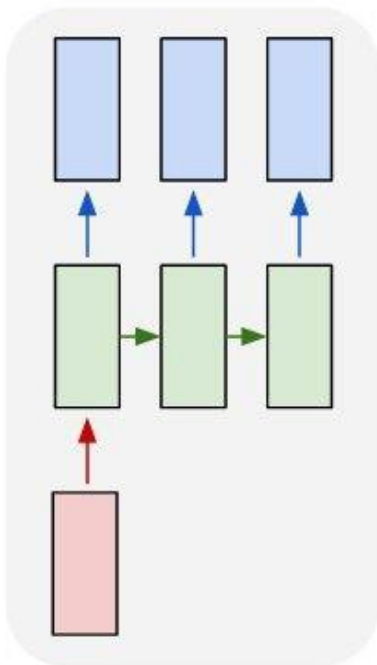
Vanilla Neural Networks

Processing Sequences

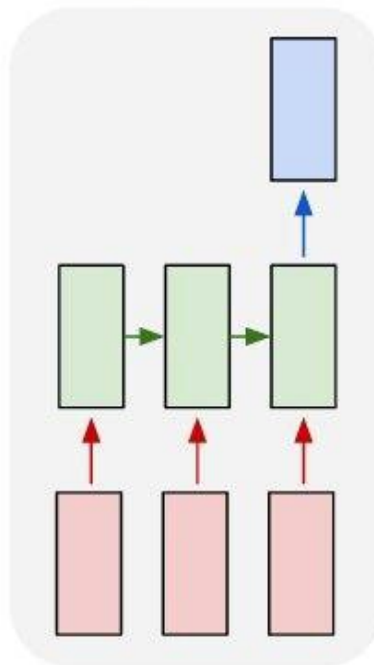
one to one



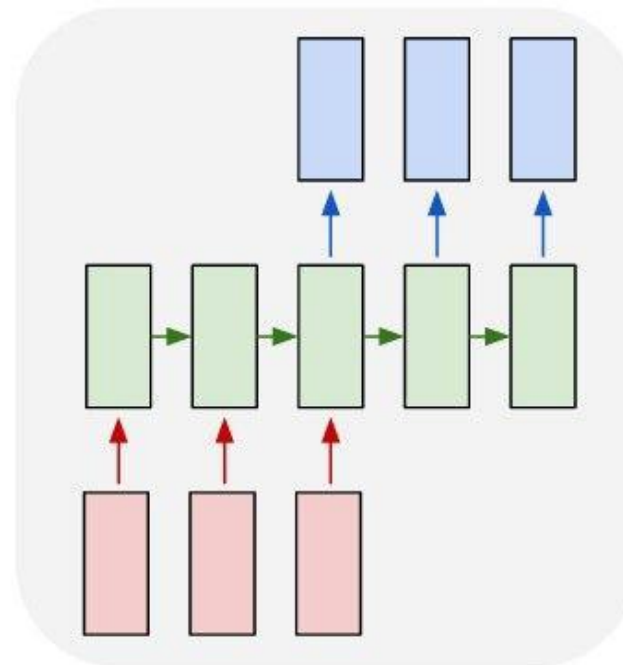
one to many



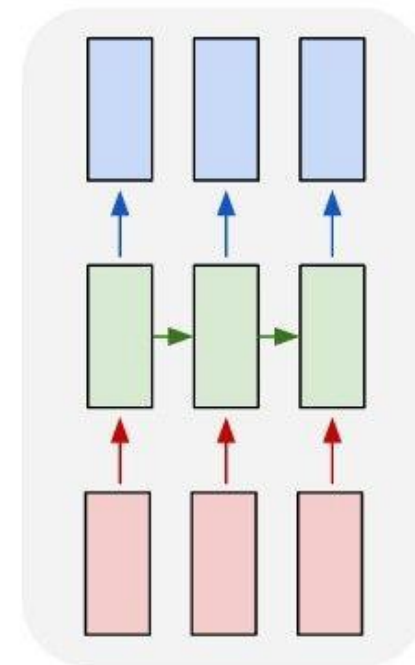
many to one



many to many



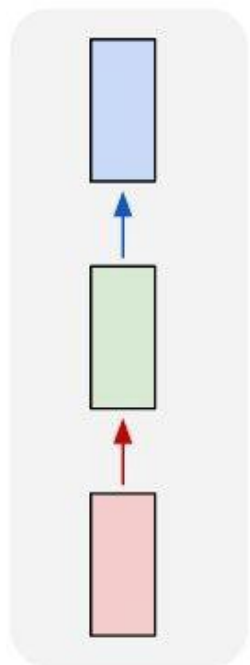
many to many



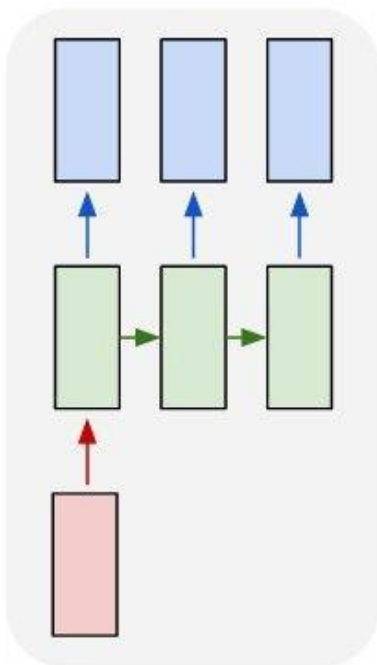
↖ e.g. **Image Captioning**
image -> sequence of words

Processing Sequences

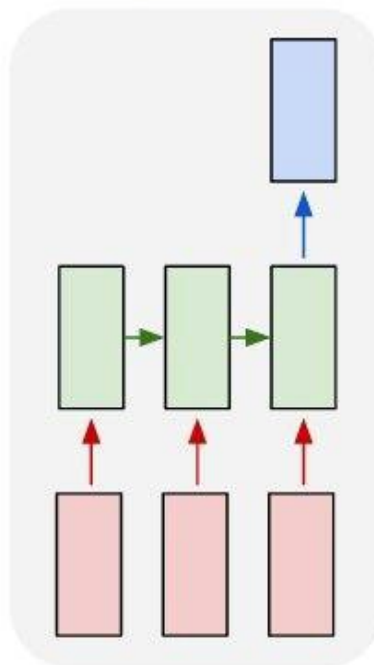
one to one



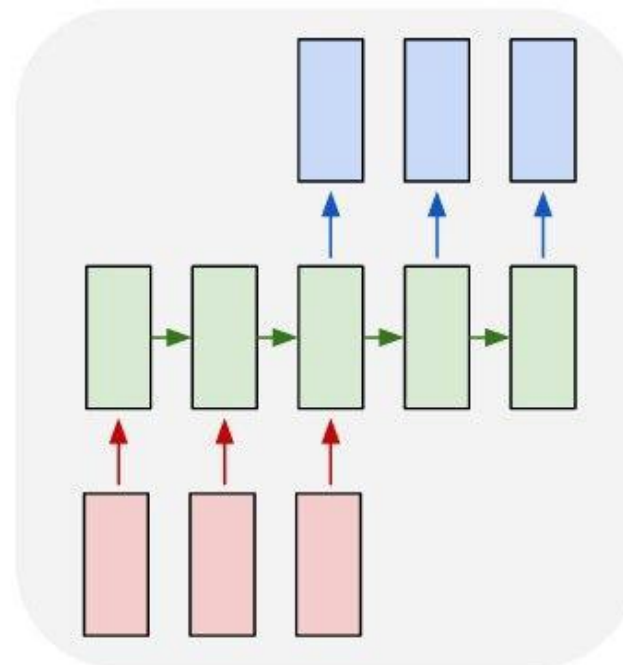
one to many



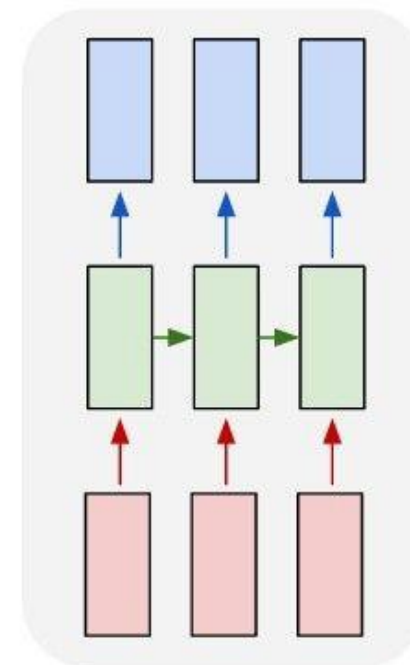
many to one



many to many



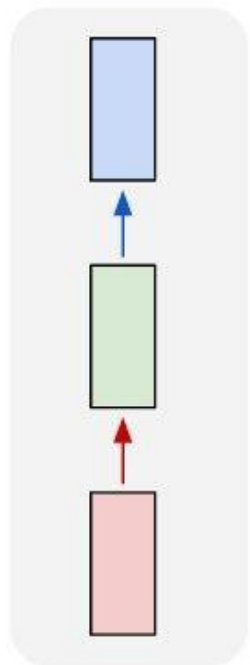
many to many



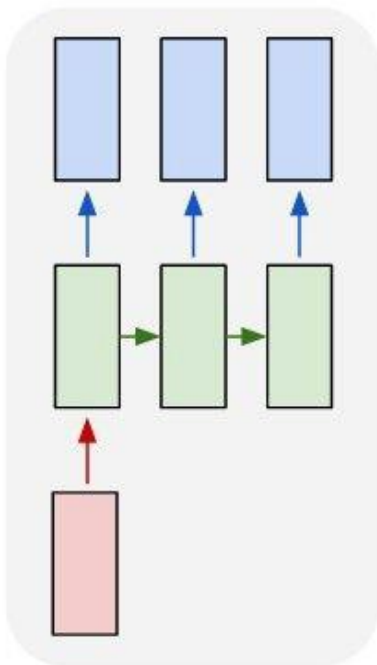
e.g. **Sentiment Classification**
sequence of words -> sentiment

Processing Sequences

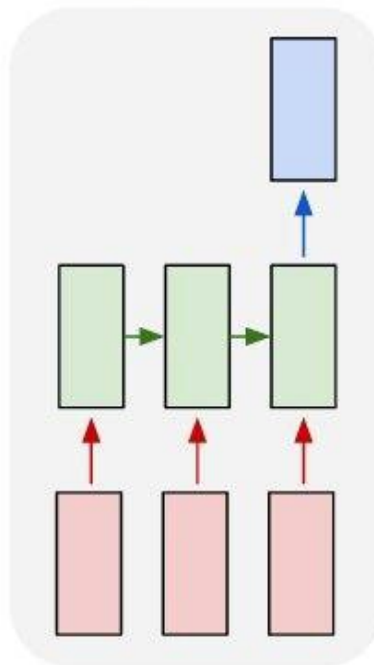
one to one



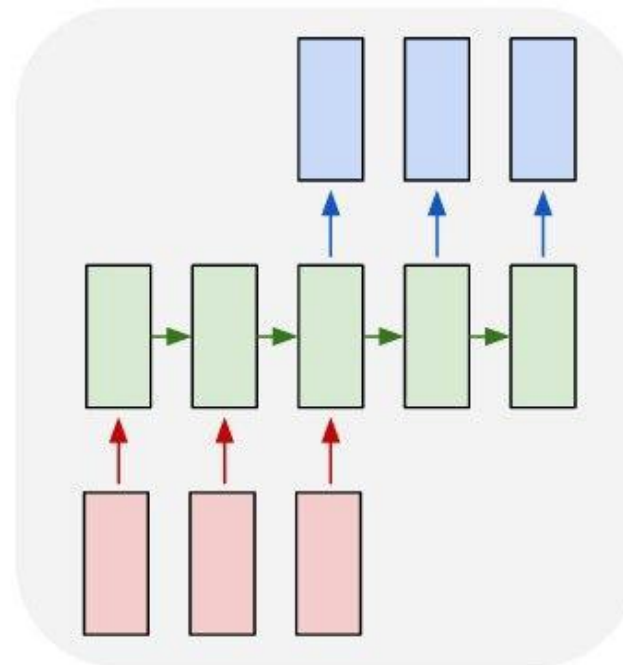
one to many



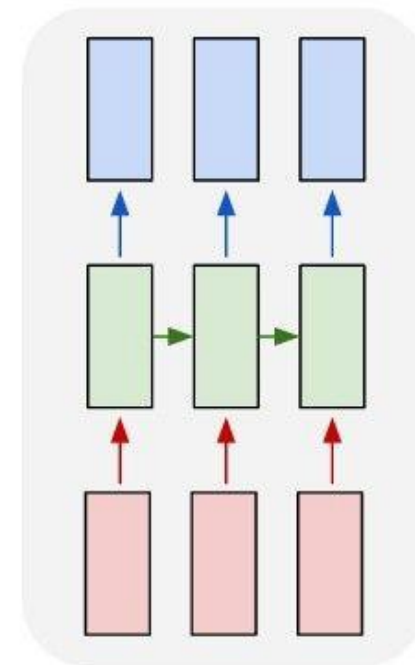
many to one



many to many



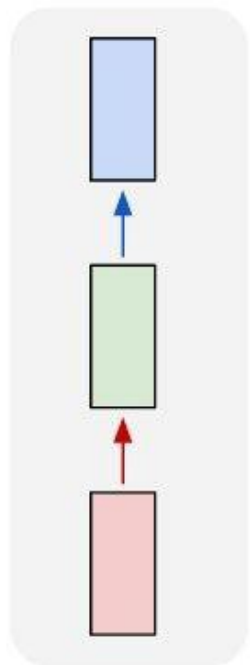
many to many



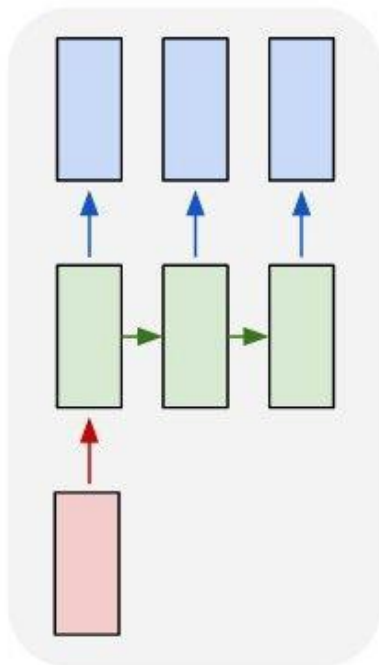
↖ e.g. **Machine Translation**
seq of words -> seq of words

Processing Sequences

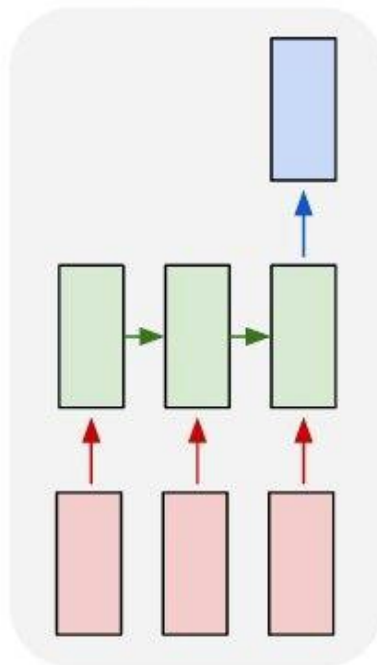
one to one



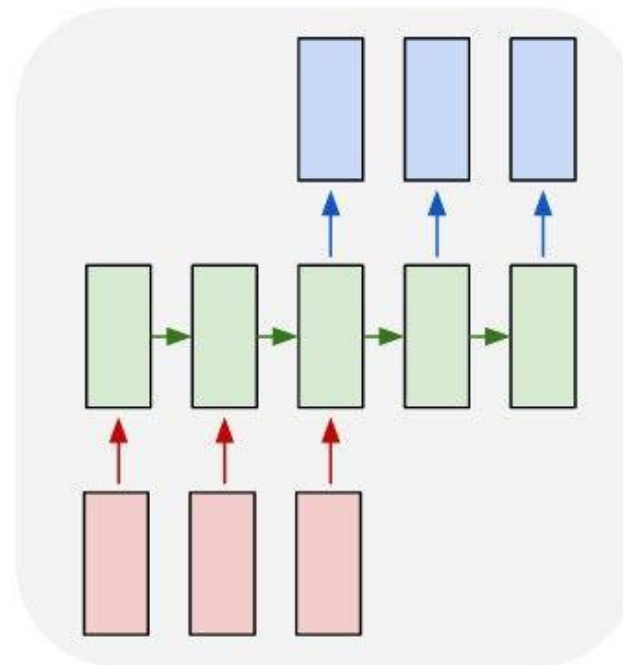
one to many



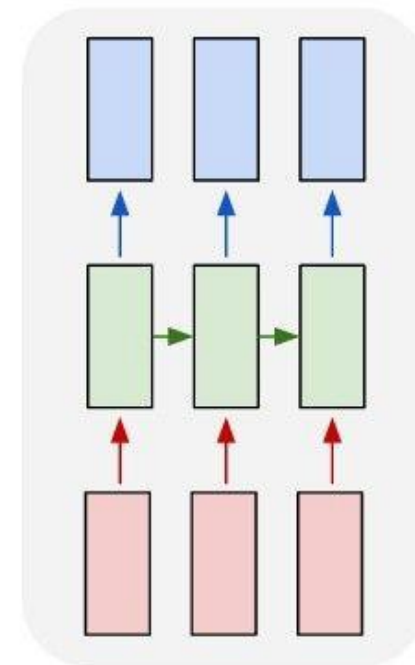
many to one



many to many



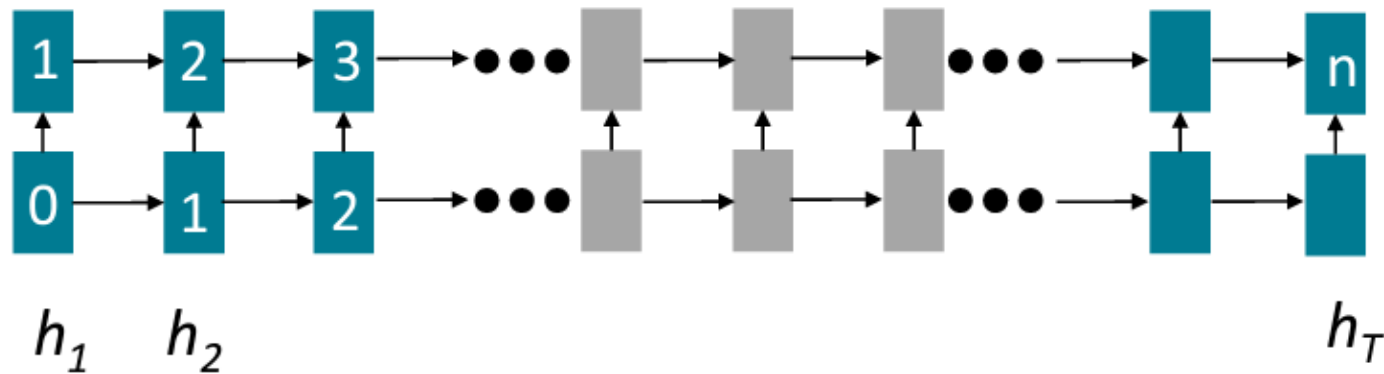
many to many



e.g. **Video classification on frame level**



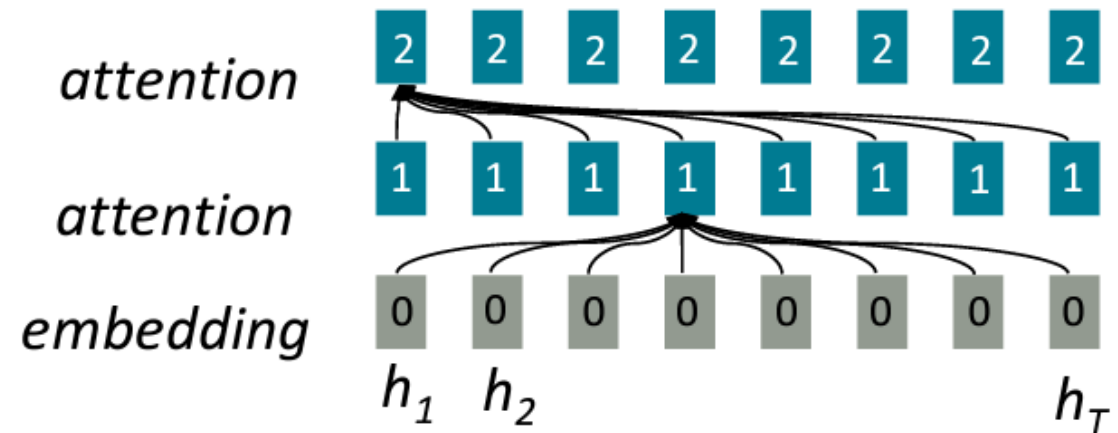
Problems with Previous Methods: Lack of Parallelizability



Compute all hidden states sequentially

Prevents training on very large datasets!

Instead, we could have another architecture that allows more **parallel computation** → *self-attention*



Transformer for machine translation

We will use machine translation for explaining transformers!

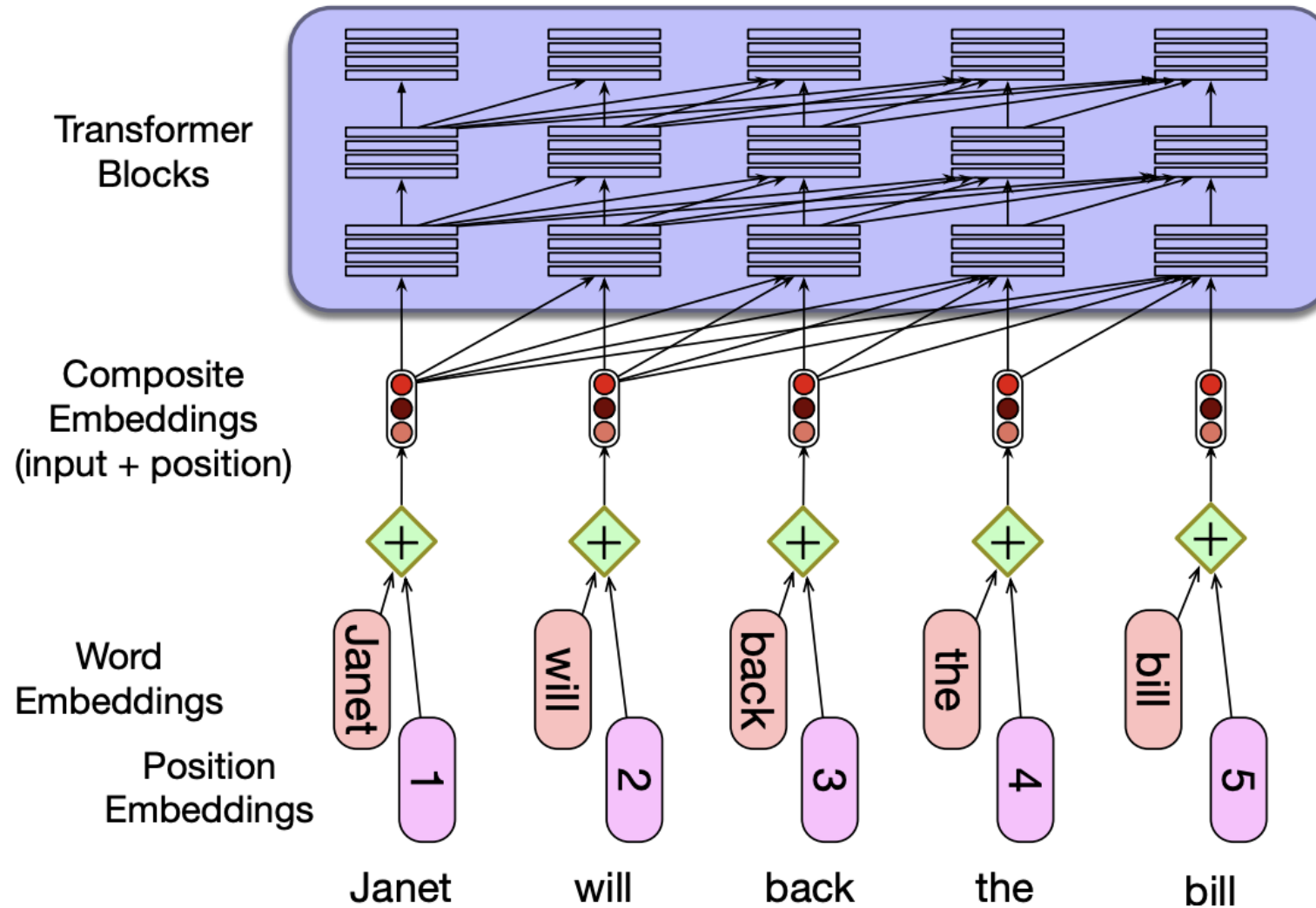


Processing input to transformer

- We will convert words to use **word embeddings**
 - Numeric representations of each word (one unique per word)
- It's important to include where a word occurs in a sentence
 - We will add **positional embeddings** to each word embedding
- Input to transformer:
$$\text{composite embedding} = \text{word embedding} + \text{positional embedding}$$

Input

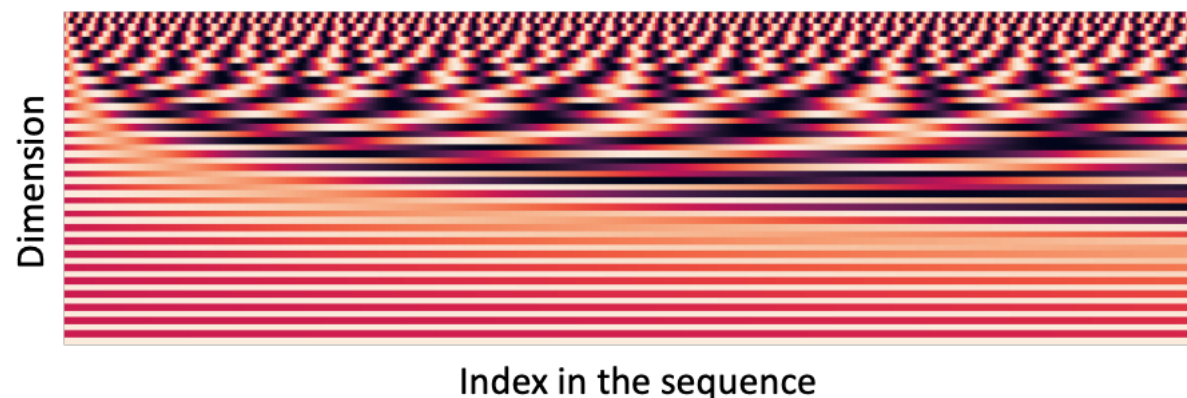
Embeddings are a means of representing objects like text, images and audio as points in a continuous vector space. Ideally, they capture the semantics or other meaningful pieces of knowledge.



Sinusoidal position representations

- Concatenate sinusoidal functions of varying periods:

$$\mathbf{p}_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



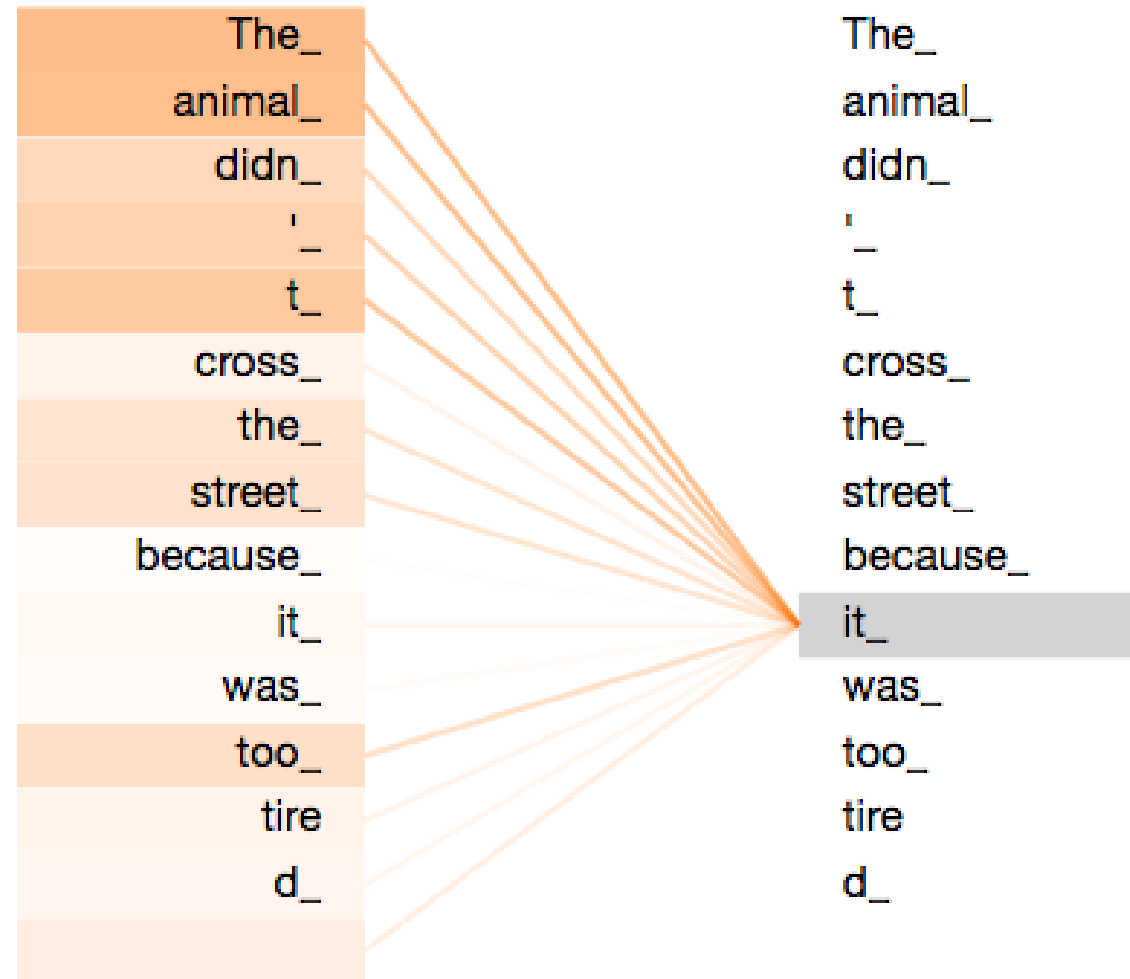
- Property of sinusoids: for any fixed offset k , $\mathbf{p}(i+k)$ is a linear function (a rotation) of $\mathbf{p}(i)$, so attention can learn relative positions easily.
 - For details, see <https://timodenk.com/blog/linear-relationships-in-the-transformers-positional-encoding/>

What is Attention?

- Answers the question: which part of the input should we focus on?
- Suppose I gave you a machine learning textbook and asked you to give me information about categorical cross-entropy. Which is faster?
 - A. Read the book cover-to-cover and come back with the answer
 - B. Go to the index, find the 'losses' chapter, go to the cross-entropy part and read the part of Categorical Cross Entropy

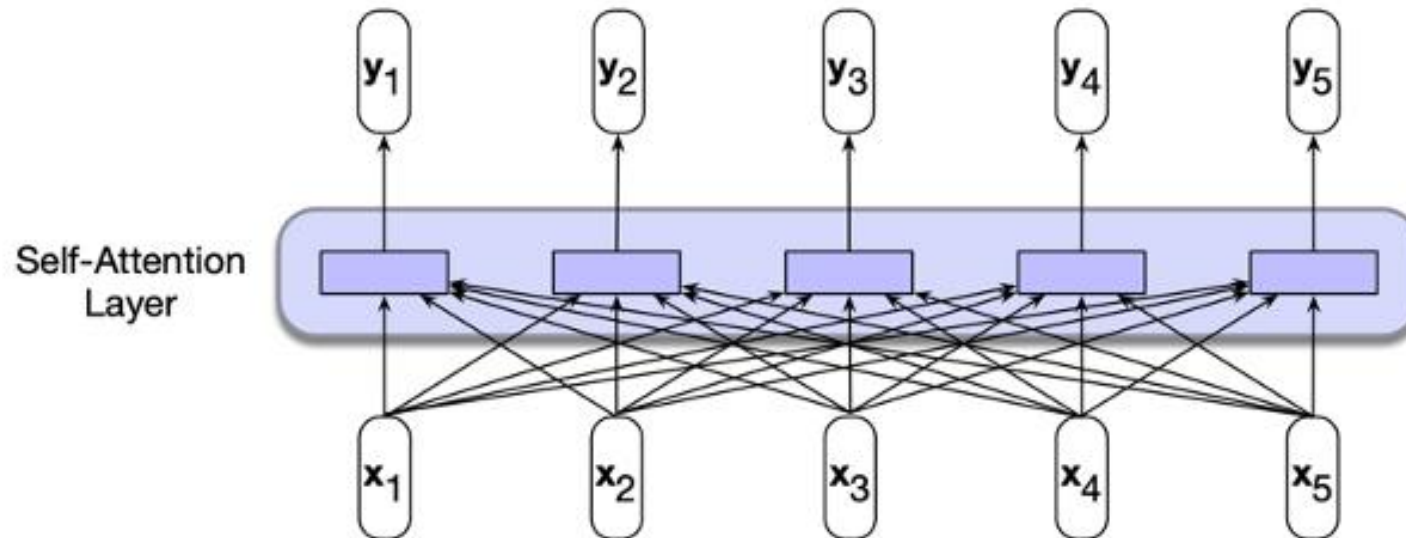
Attention in Text

“The animal didn't cross the street because it was too tired”



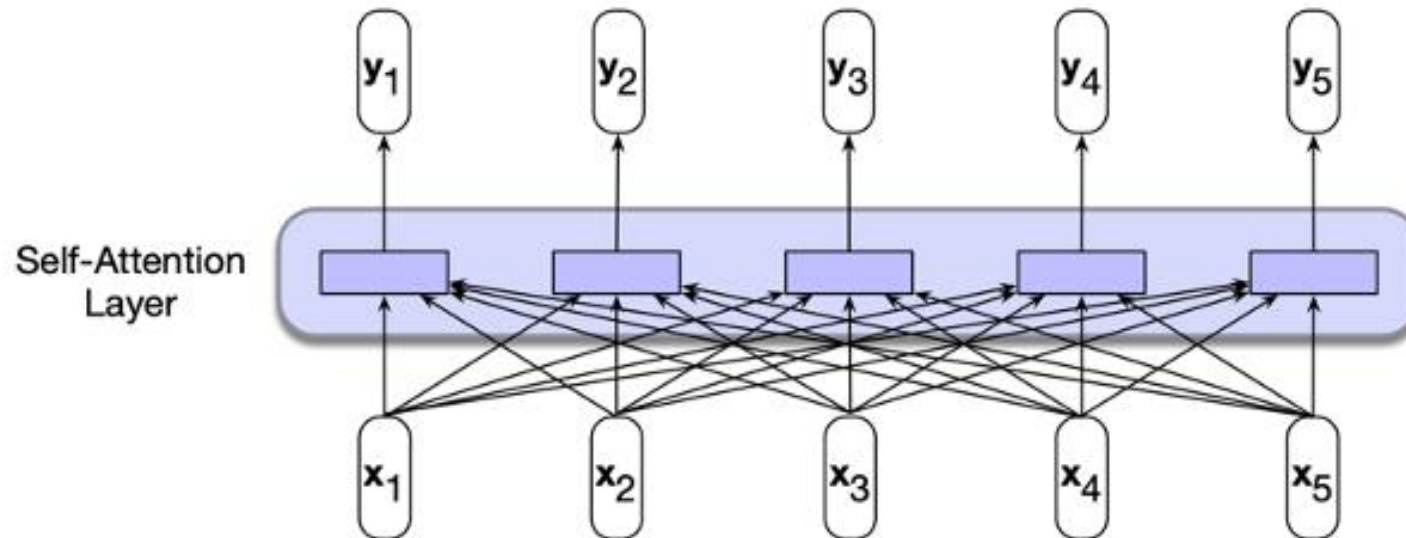
Self-attention

- A way for the network to directly extract and use information from arbitrarily large contexts
- A single self-attention layer maps input sequences $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ to output sequences $(\mathbf{y}_1, \dots, \mathbf{y}_n)$ of the same length

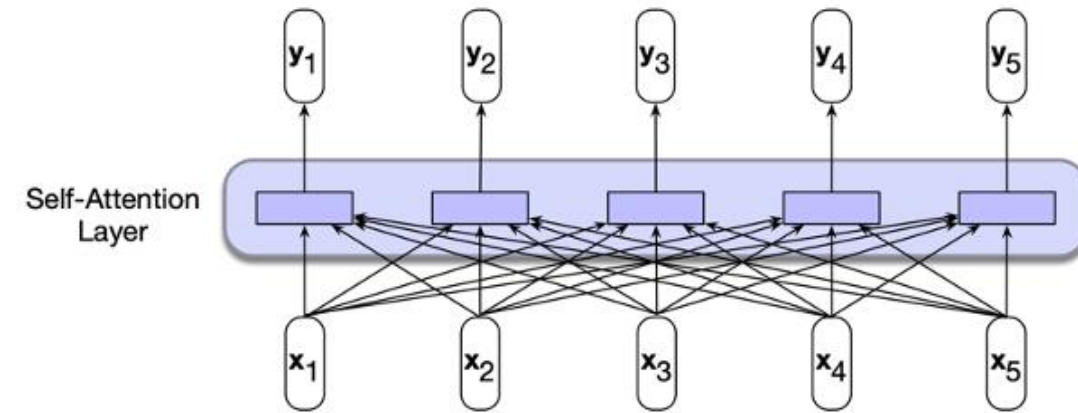


Self-attention

- When processing a single input item:
 - The model considers all other input items, including the current input itself
 - Computation is independent of all other computations (easy to parallelize!)



Self-attention



- Let's consider input $\mathbf{x}_3$:
 - We want to compare $\mathbf{x}_3$ to all previous & current inputs, to figure out what the most relevant inputs are
 - We'll use the **dot product** to do the comparisons
 - Comparisons: $\mathbf{x}_3 \cdot \mathbf{x}_1$ $\mathbf{x}_3 \cdot \mathbf{x}_2$ $\mathbf{x}_3 \cdot \mathbf{x}_3$
 $\mathbf{x}_3 \cdot \mathbf{x}_4$ $\mathbf{x}_3 \cdot \mathbf{x}_5$
 - In general: $score(\mathbf{x}_3, \mathbf{x}_j) = \mathbf{x}_3 \cdot \mathbf{x}_j$
- Next, we want to turn this into a probability distribution! We can use the softmax function

$$\alpha_{3,j} = \text{softmax}\left(score(\mathbf{x}_3, \mathbf{x}_j)\right) \forall j$$

Aside: Softmax

- For a vector $\mathbf{z}$ with k dimensions:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

- The softmax of an input vector $\mathbf{z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_k]$ is also a vector:

$$\text{softmax}(\mathbf{z}) = \left[\frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

Aside: Softmax

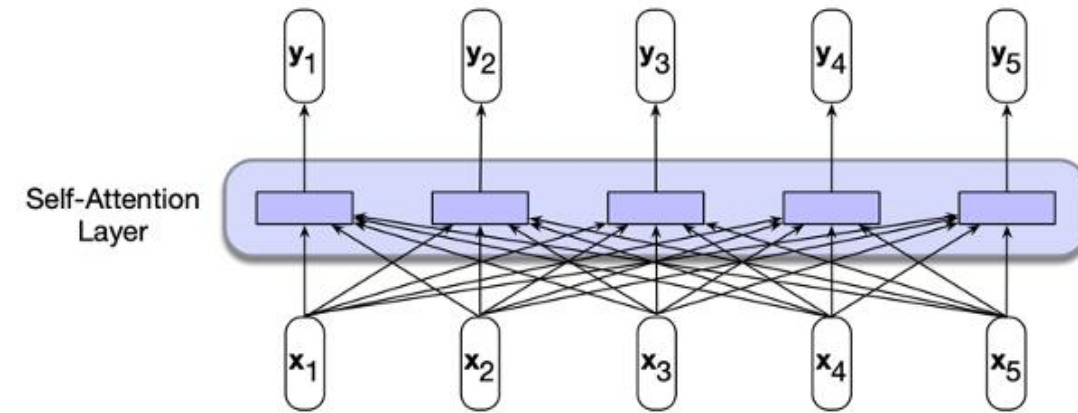
- Like the sigmoid, softmax squashes values toward 0 or 1
 - If one of the inputs is larger than the others, it will be pushed towards 1
 - Other inputs will be pushed towards 0
- What is the softmax of $\mathbf{z} = [0.6, 1.1, -1.5]$?

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

When poll is active respond at [PollEv.com/492](https://pollev.com/492)



Back to Self-attention



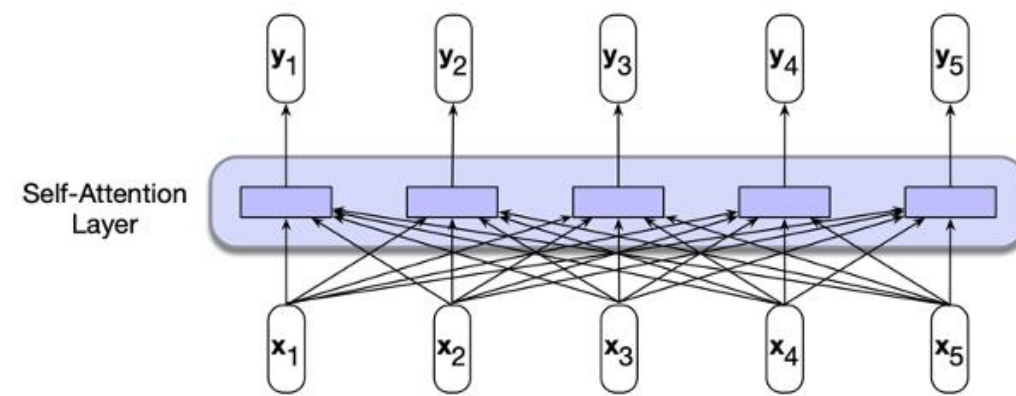
- Let's consider input $\mathbf{x}_3$:
 - Softmax gave us a vector of weights α_{ij} that indicate the proportional relevance of each input to $\mathbf{x}_3$

$$\alpha_{3,j} = \text{softmax}(\text{score}(\mathbf{x}_3, \mathbf{x}_j)) \forall j$$

- We can use these weights to generate an output value $\mathbf{y}_i$ by taking the sum of the inputs seen so far, weighted by their respective α value

$$\mathbf{y}_3 = \sum_j \alpha_{3,j} \mathbf{x}_j$$

Self-attention



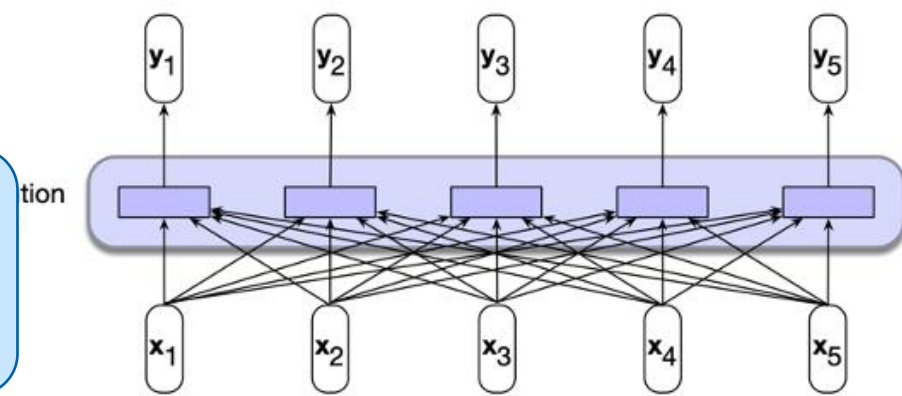
- These equations can be generalized to any input i

$$\begin{aligned} \text{score}(\mathbf{x}_i, \mathbf{x}_j) &= \mathbf{x}_i \cdot \mathbf{x}_j \\ \alpha_{i,j} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \\ \mathbf{y}_i &= \sum_{j} \alpha_{ij} \mathbf{x}_j \end{aligned}$$

- This is the core of attention:
 - A set of comparisons to relevant items in some context
 - A normalization of those scores to provide a probability distribution
 - A weighted sum using this distribution

Self-attention

Where do we use the input embedding x_i in these calculations?



- These equations can be generalized to any input i

$$score(x_i, x_j) = x_i \cdot x_j$$

$$\alpha_{i,j} = softmax(score(x_i, x_j)) \quad \forall j$$

query

1. As the *current focus of attention* when being compared to all of the other inputs

attention:

- A set of comparisons to relevant items in some context
- A normalization of those scores to provide a probability
- A weighted sum using this distribution

$$y_i = \sum_{j} \alpha_{ij} x_j$$

key

2. In its role as an *input* being compared to the current focus of attention

3. As a *value* used to compute the output for the current focus of attention

value

Self-attention

- Each input embedding $\mathbf{x}_i$ takes on three different roles: **query**, **key**, and **value**
- We will use weight matrices to project $\mathbf{x}_i$ into a representation of its role for each:

$$\begin{aligned}\mathbf{q}_i &= \mathbf{W}^Q \mathbf{x}_i \\ \mathbf{k}_i &= \mathbf{W}^K \mathbf{x}_i \\ \mathbf{v}_i &= \mathbf{W}^V \mathbf{x}_i\end{aligned}$$

Self-attention

- We can update our equations with our new projected embeddings
- Need to update score equation:
 - Previously: $score(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$
 - Now: $score(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{q}_i \cdot \mathbf{k}_j$
- Need to update weighted sum:
 - Previously: $\mathbf{y}_i = \sum_{ij} \alpha_{ij} \mathbf{x}_j$
 - Now: $\mathbf{y}_i = \sum_{ij} \alpha_{ij} \mathbf{v}_j$

One more problem!
Dot products can give us arbitrarily large values (positive or negative). This can cause problems with the gradients during training.

Solution: Scale the dot product.
One typical approach:
$$score(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$
where d_k is the dimensions of the $\mathbf{q}_i$ and $\mathbf{k}_j$ vectors

Self-attention: putting it all together

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i$$

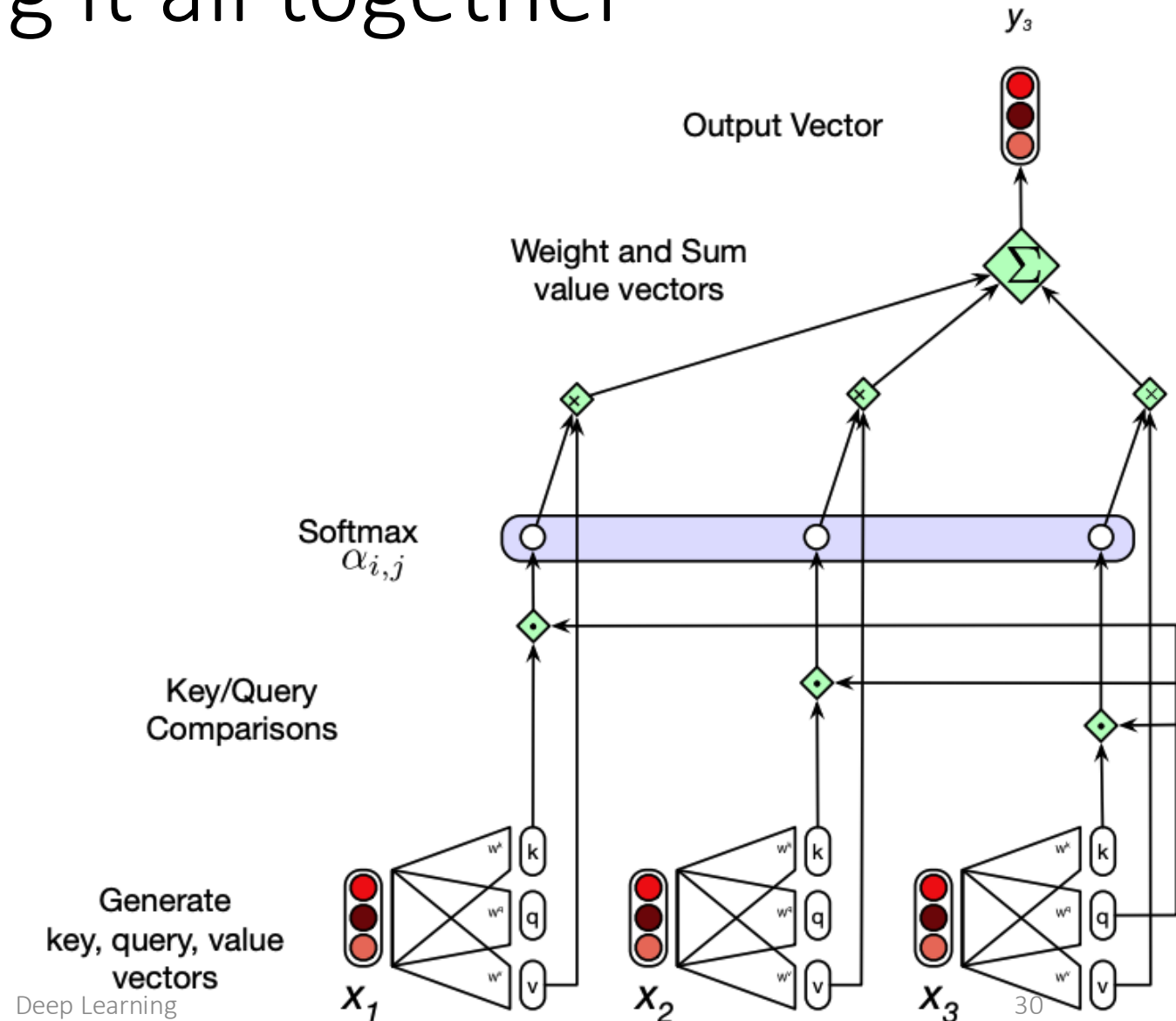
$$\mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i$$

$$\mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j$$
$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j$$

$$\mathbf{y}_i = \sum_{j} \alpha_{ij} \mathbf{v}_j$$



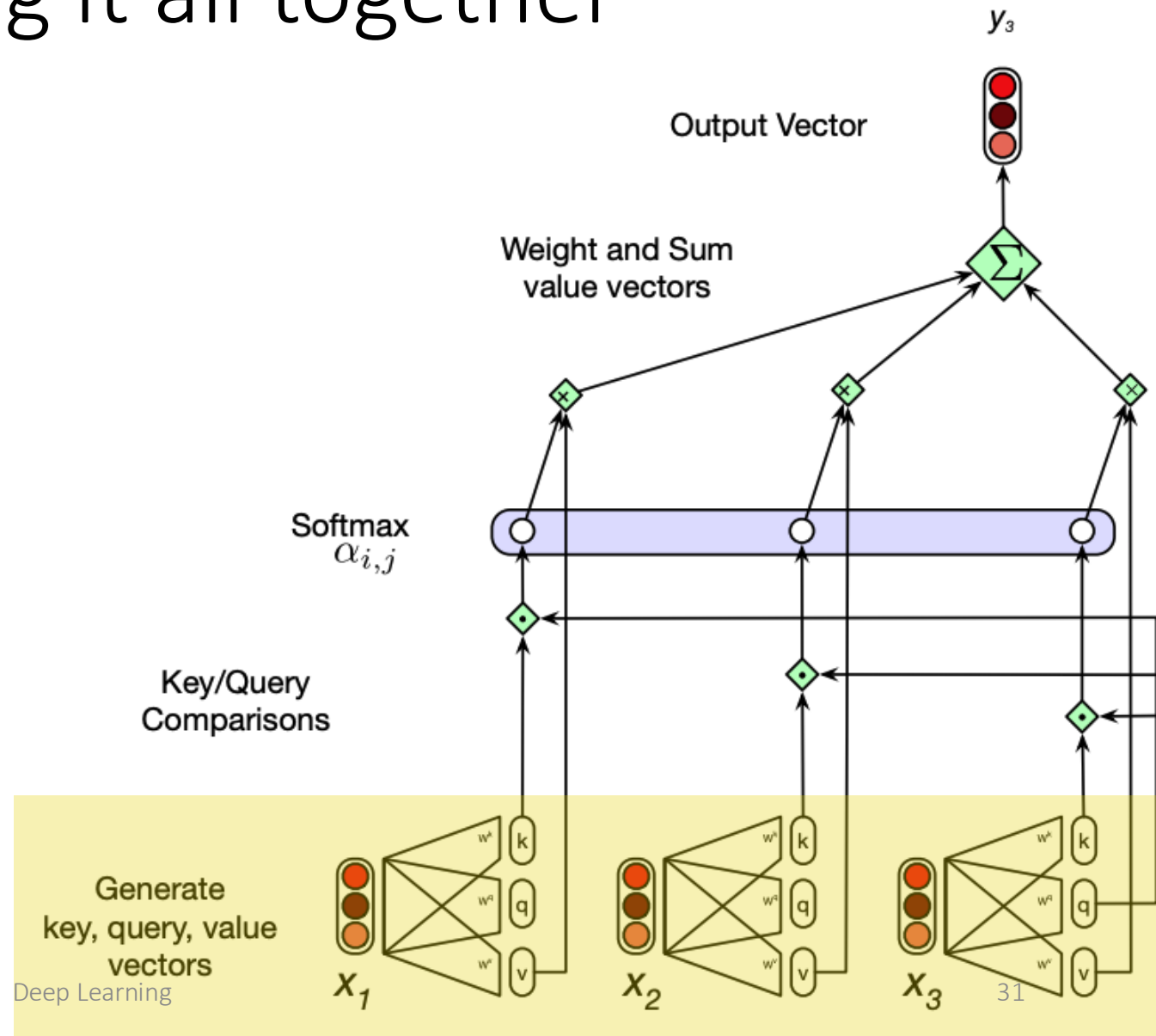
Self-attention: putting it all together

$$\begin{aligned} \mathbf{q}_i &= \mathbf{W}^Q \mathbf{x}_i \\ \mathbf{k}_i &= \mathbf{W}^K \mathbf{x}_i \\ \mathbf{v}_i &= \mathbf{W}^V \mathbf{x}_i \end{aligned}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\begin{aligned} \alpha_{i,j} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \end{aligned}$$

$$\mathbf{y}_i = \sum_{j} \alpha_{ij} \mathbf{v}_j$$



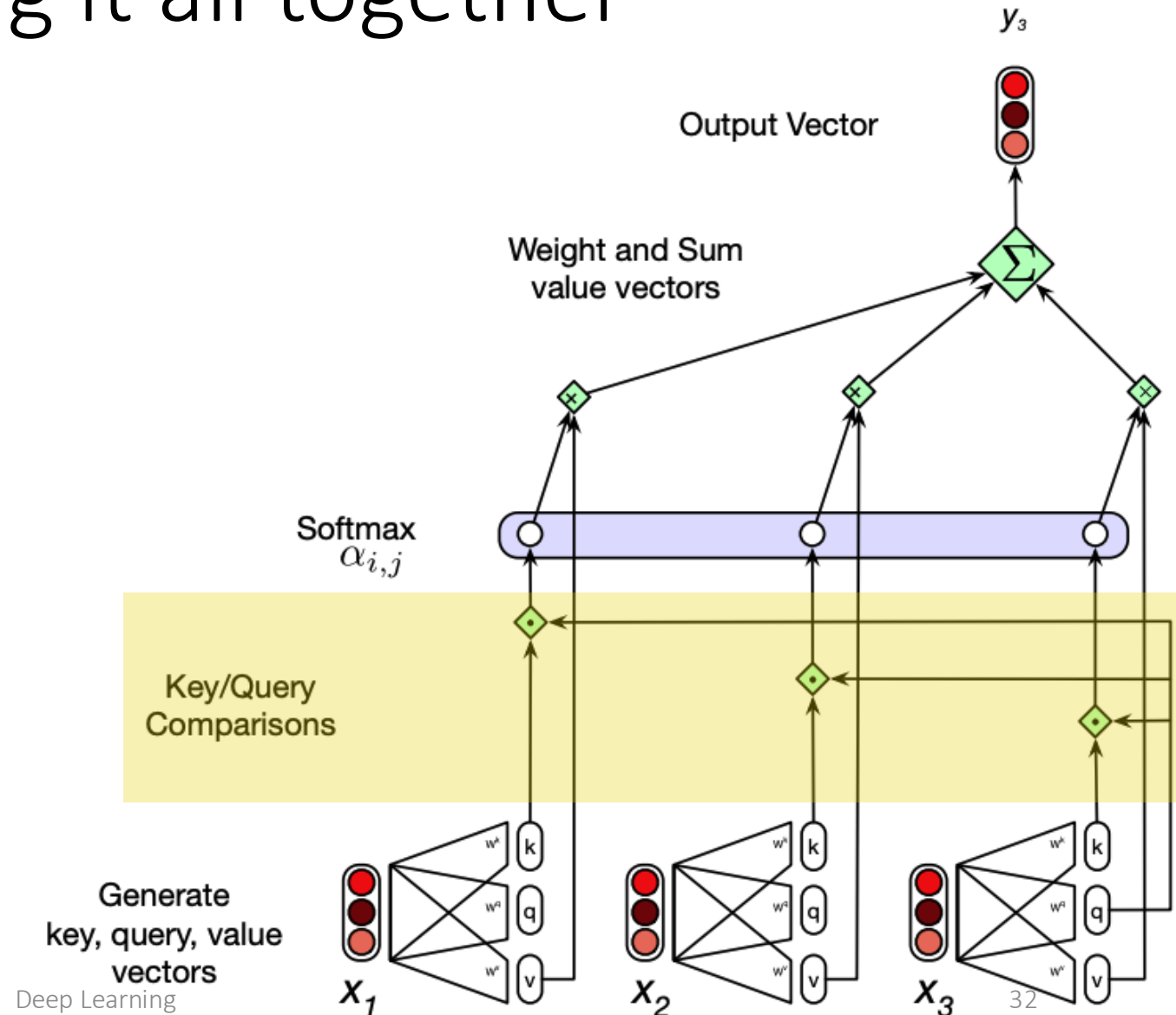
Self-attention: putting it all together

$$\begin{aligned} \mathbf{q}_i &= \mathbf{W}^Q \mathbf{x}_i \\ \mathbf{k}_i &= \mathbf{W}^K \mathbf{x}_i \\ \mathbf{v}_i &= \mathbf{W}^V \mathbf{x}_i \end{aligned}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\begin{aligned} \alpha_{i,j} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \end{aligned}$$

$$\mathbf{y}_i = \sum_{j} \alpha_{ij} \mathbf{v}_j$$



Self-attention: putting it all together

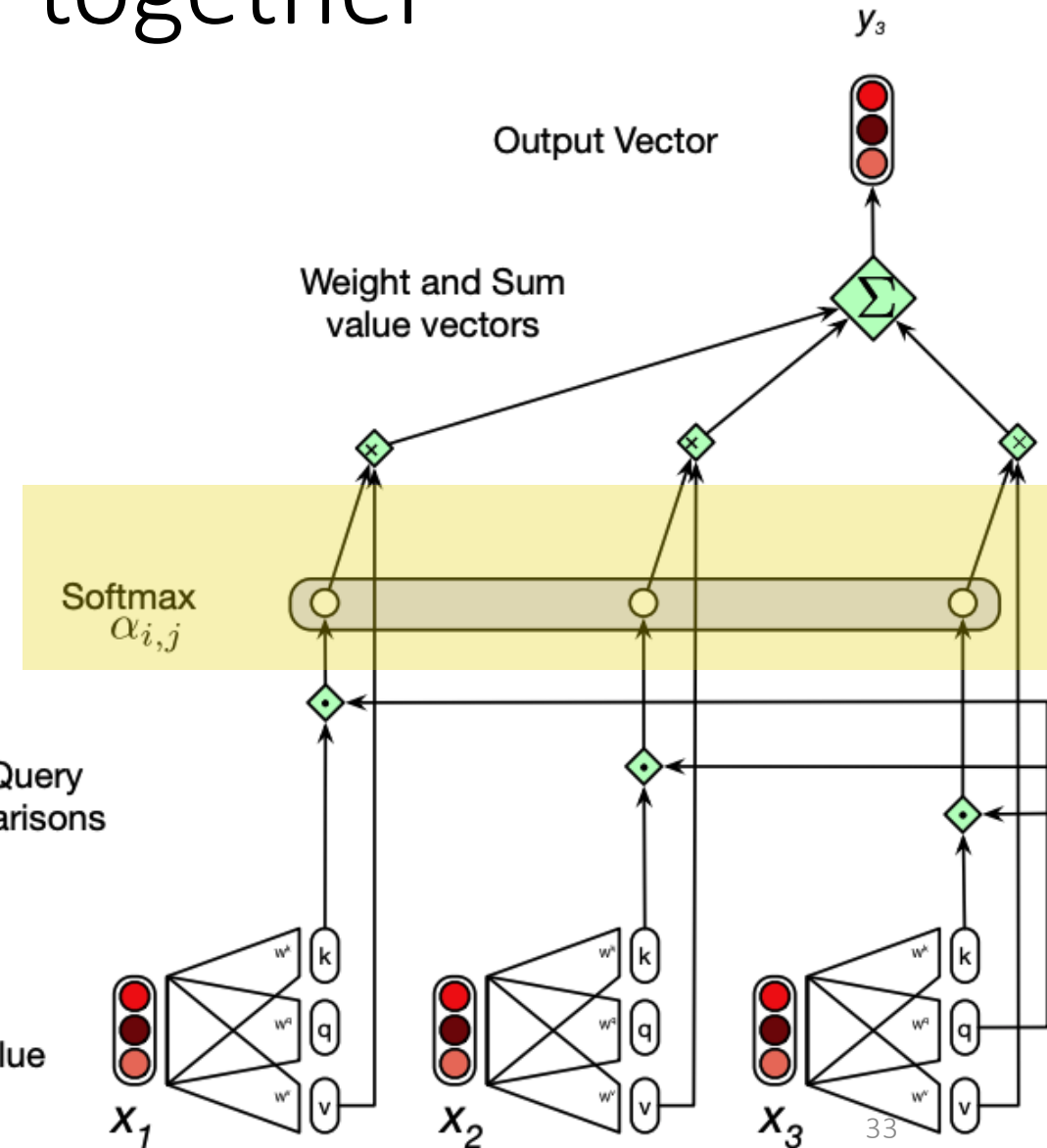
$$\begin{aligned} \mathbf{q}_i &= \mathbf{W}^Q \mathbf{x}_i \\ \mathbf{k}_i &= \mathbf{W}^K \mathbf{x}_i \\ \mathbf{v}_i &= \mathbf{W}^V \mathbf{x}_i \end{aligned}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\begin{aligned} \alpha_{i,j} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \end{aligned}$$

$$\mathbf{y}_i = \sum_{j} \alpha_{ij} \mathbf{v}_j$$

Generate
key, query, value
vectors
Deep Learning



Self-attention: putting it all together

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i$$

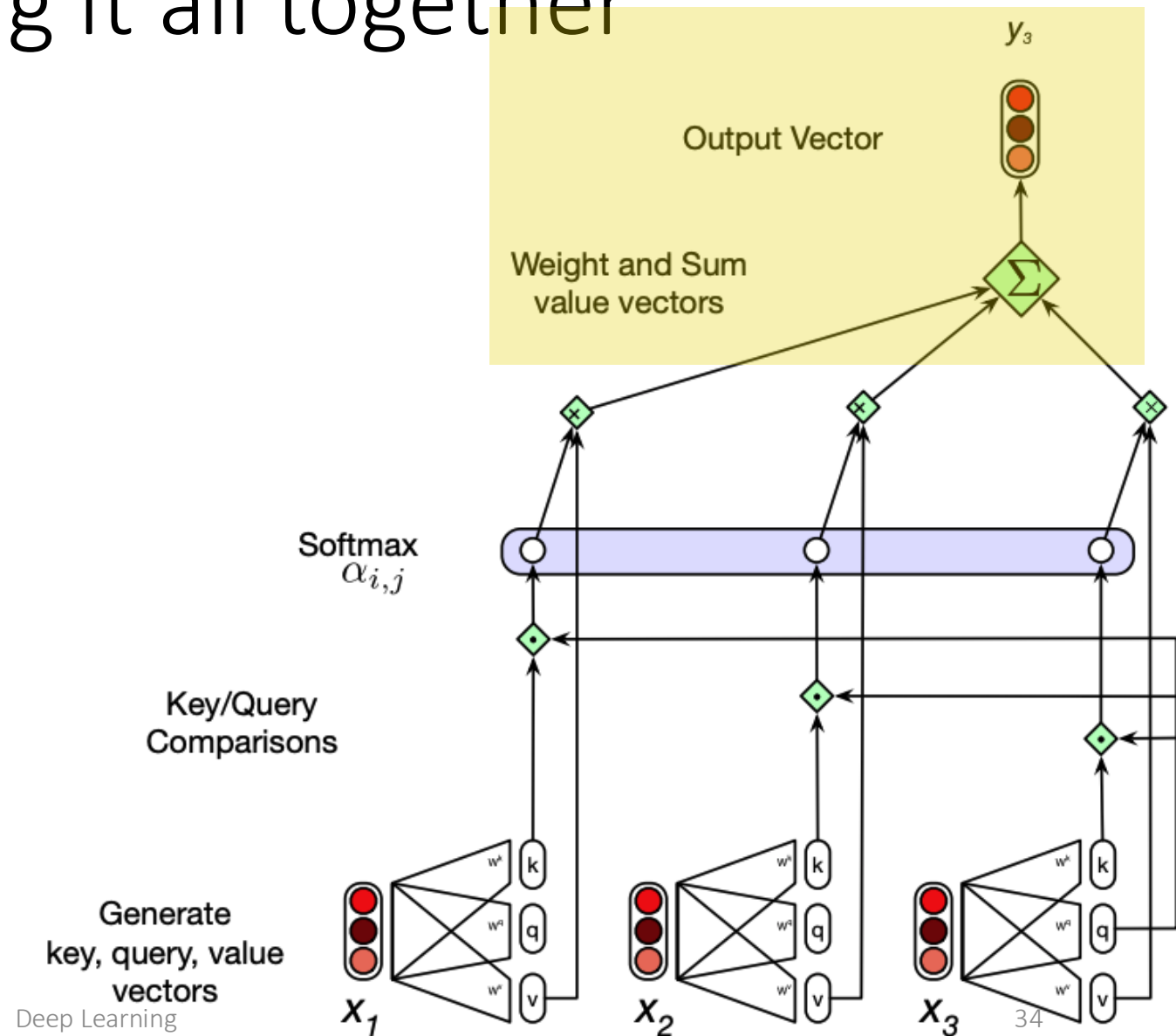
$$\mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i$$

$$\mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{i,j} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j$$
$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_k \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j$$

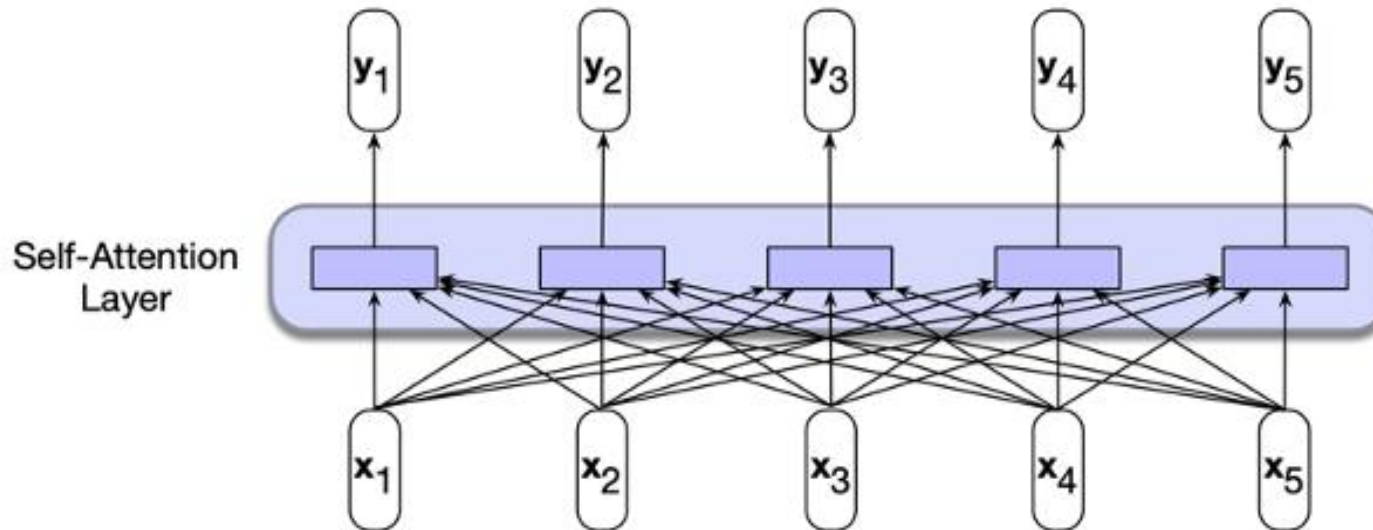
$$\mathbf{y}_i = \sum_{j} \alpha_{ij} \mathbf{v}_j$$



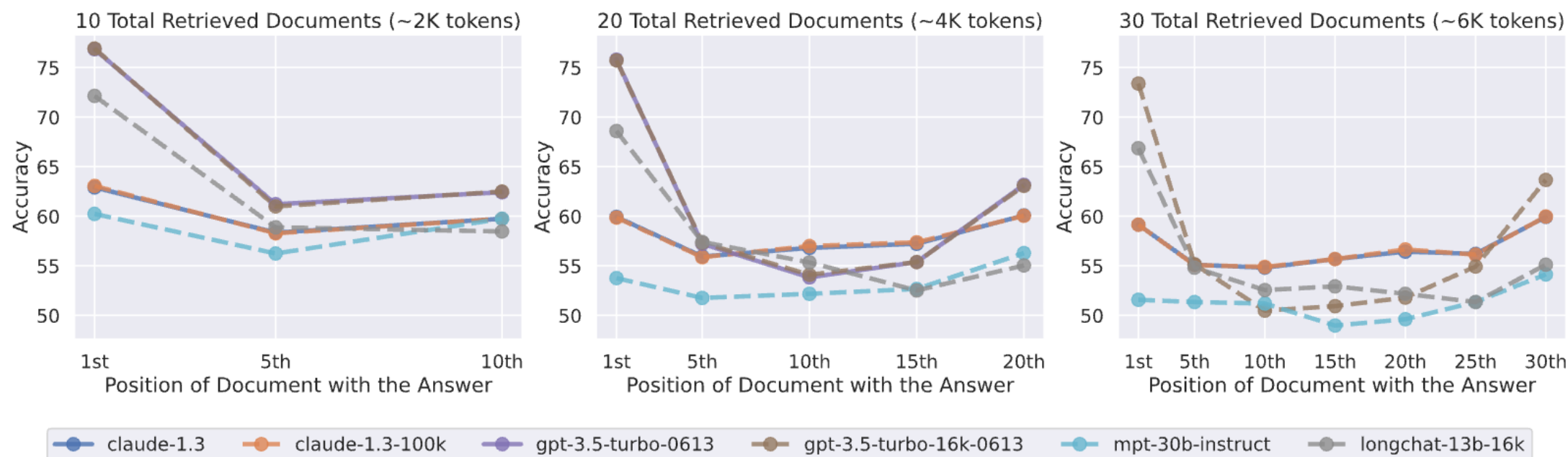
When poll is active respond at [PollEv.com/492](https://pollev.com/492)



What are some limitations of self-attention?

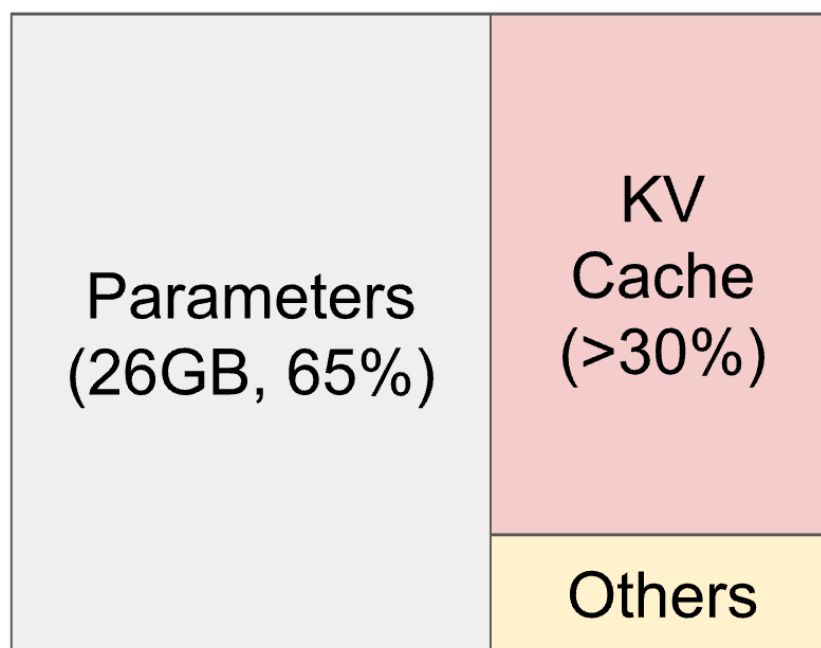


Issues w/ long-range dependencies

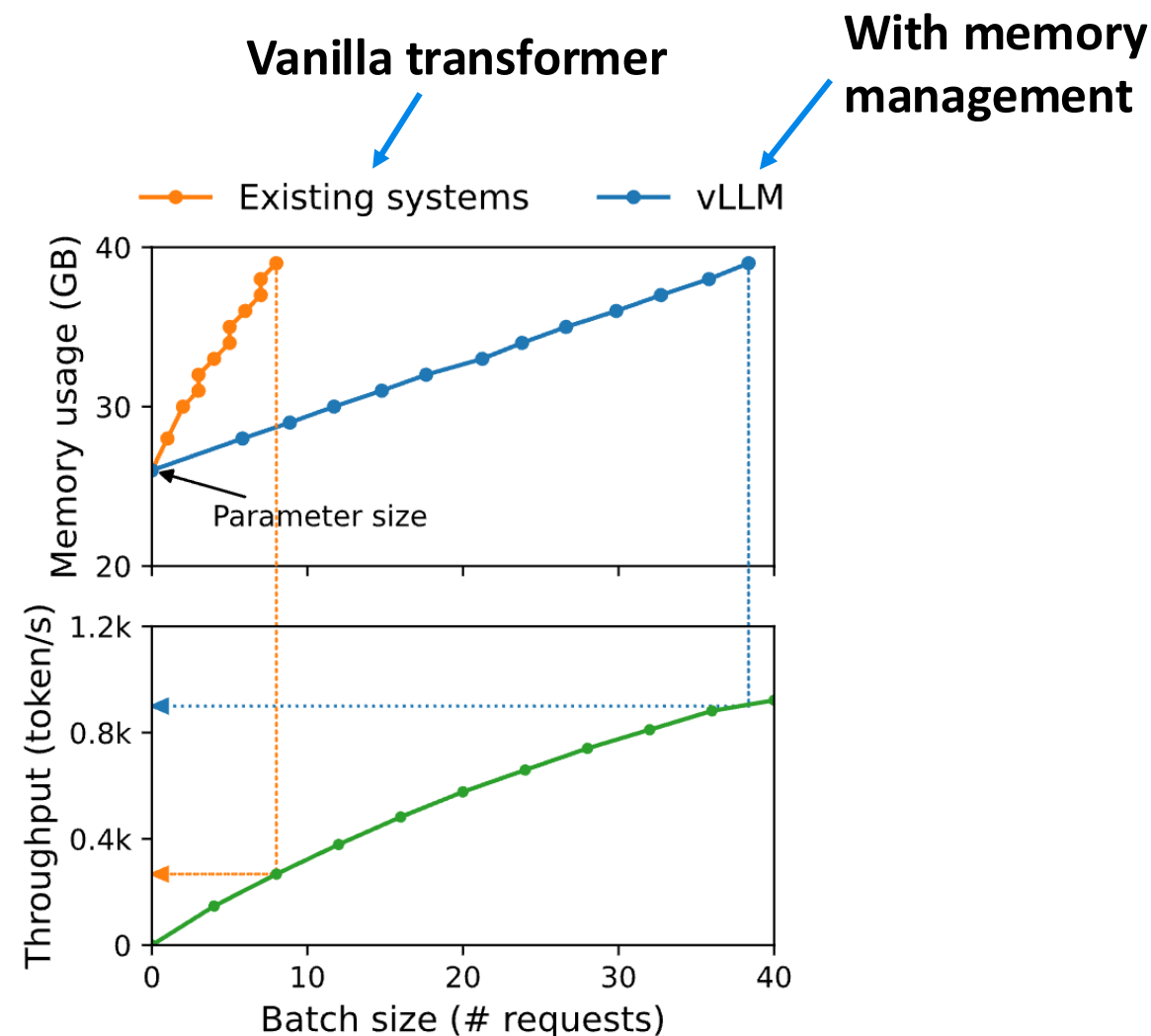


Accuracy traces a **U-shape**: it is high when the answer is first or last, and it drops sharply, by more than 20 points for some models, when the answer is in the middle.

Memory requirements

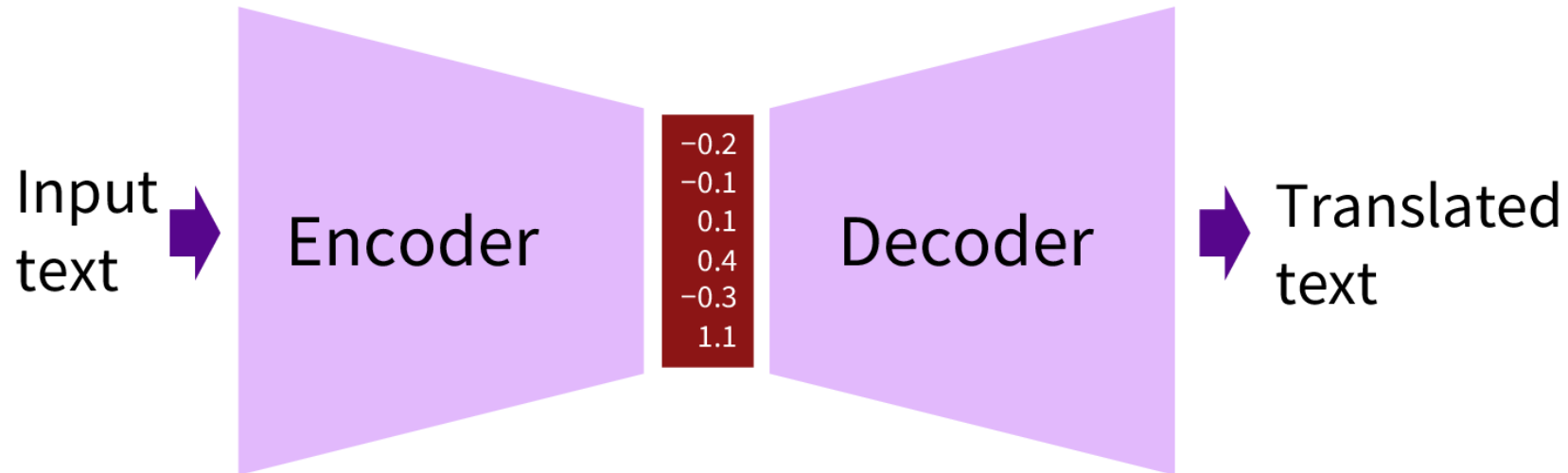


NVIDIA A100 40GB

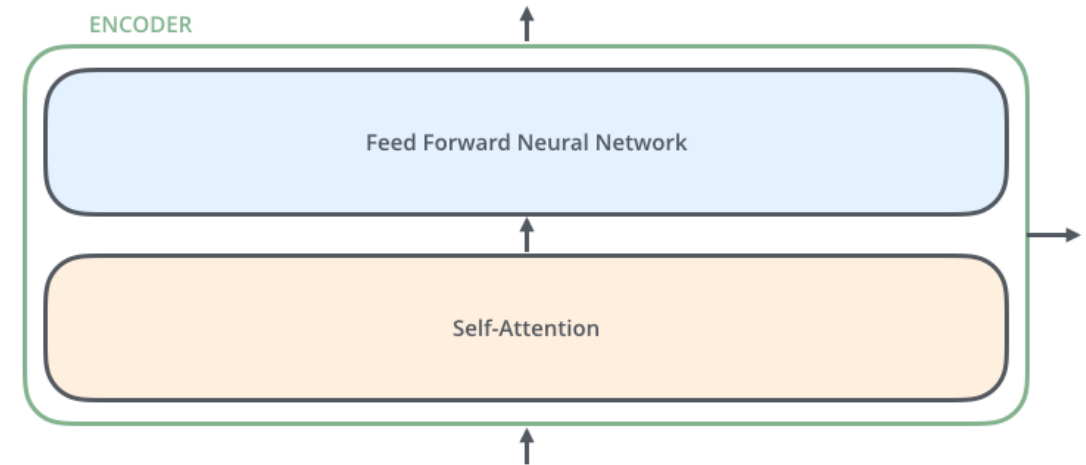
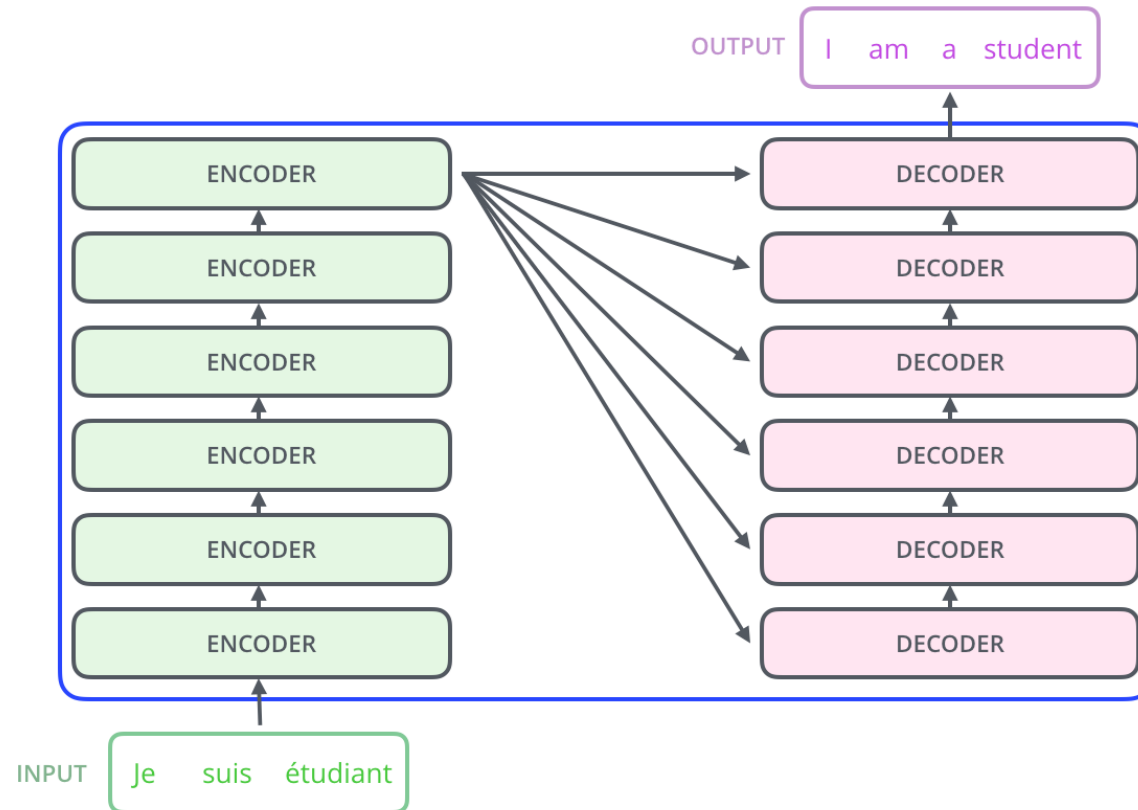


Back to transformer for machine translation

- We'll use an **encoder-decoder** architecture or **sequence-to-sequence model** (aka **seq2seq**) - needs two Transformers

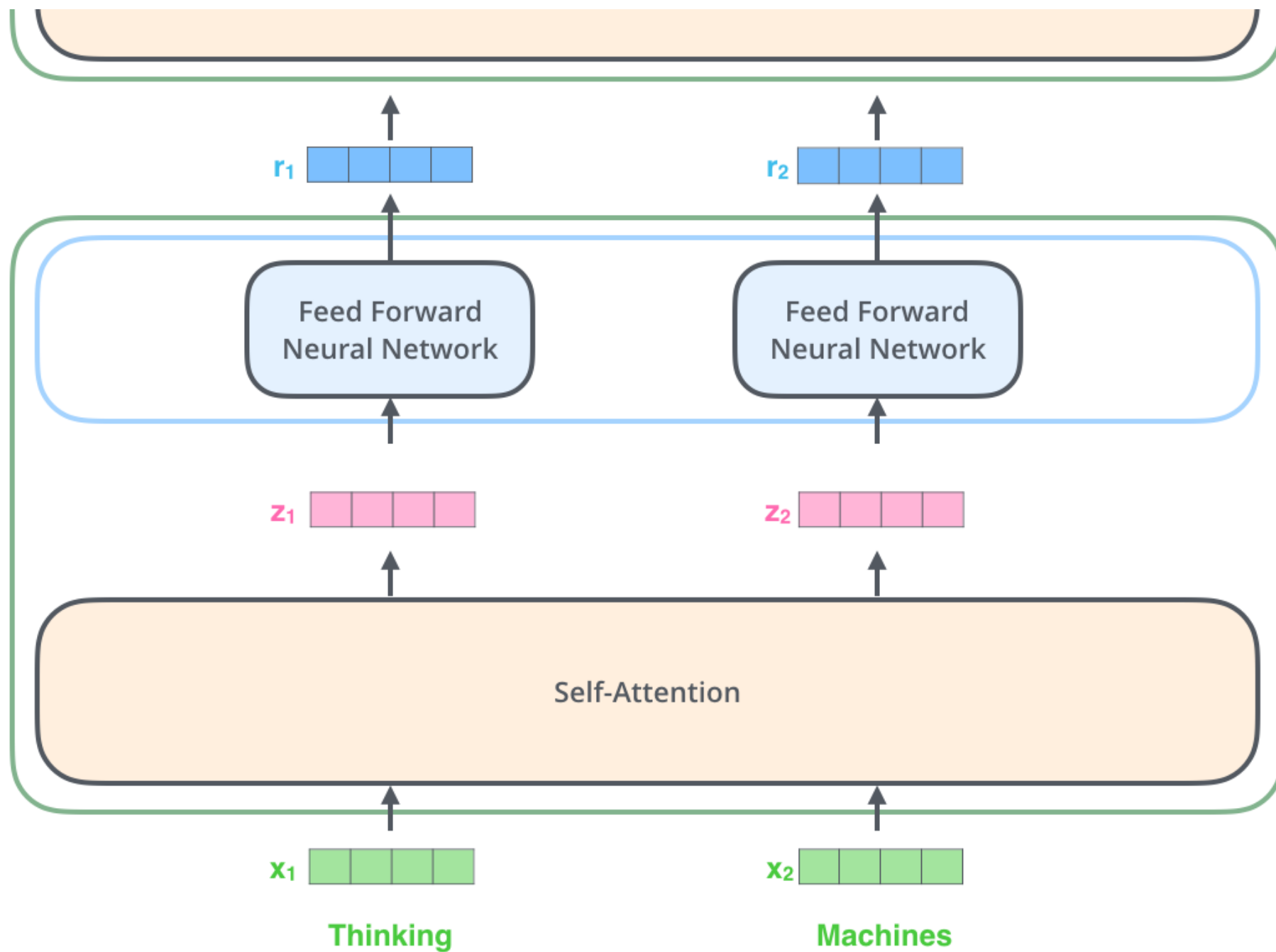


Back to transformer for machine translation



ENCODER #2

ENCODER #1

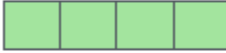


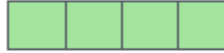
Input

Thinking

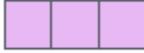
Machines

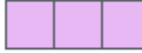
Embedding

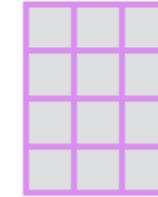
X_1 

X_2 

Queries

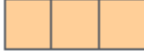
q_1 

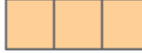
q_2 

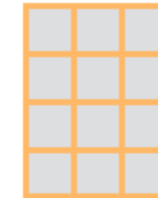


W^Q

Keys

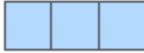
k_1 

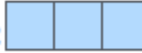
k_2 



W^K

Values

v_1 

v_2 



W^V

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^{\text{Q}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^{\text{K}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^{\text{V}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{aligned}
 & \text{softmax} \left(\frac{\begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{K}^{\text{T}} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}}{\sqrt{d_k}} \right) \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \\
 & = \begin{array}{c} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}
 \end{aligned}$$

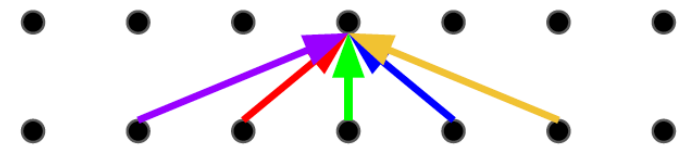
Multi-head Attention

- There are lots of different ways that words in a sentence relate or pieces of an image relate!
 - E.g., in text – relationship between subject & verb, relationship between pronoun & noun it refers to
 - It's hard for a single transformer block to capture all these relationships
- Solution: multi-head self-attention layers
 - Multiple self-attention layers (called heads) that operate in parallel
 - Each head has its own parameters (e.g., $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$)
 - Each head can learn a different aspect of the relationships between input elements
 - After all of the heads have run, the output from each head is concatenated together
 - This vector is too big! We use a linear projection to project down to dimension d

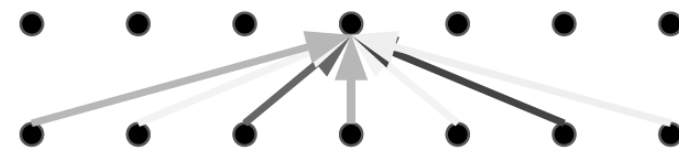
Convolution vs Self-Attention vs Multi-head Attention

- Convolution:
 - A different linear transformation for each relative position. Allows you to distinguish what information came from where.
- Self-Attention:
 - A weighted average.
- Multi-head Attention:
 - Multiple attention layers (heads) in parallel (shown by different colors). Each head uses different linear transformations. Different heads can learn different relationships.

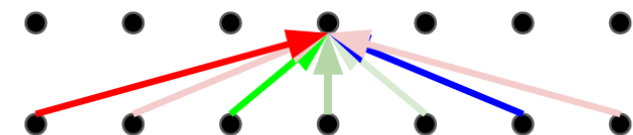
Convolution



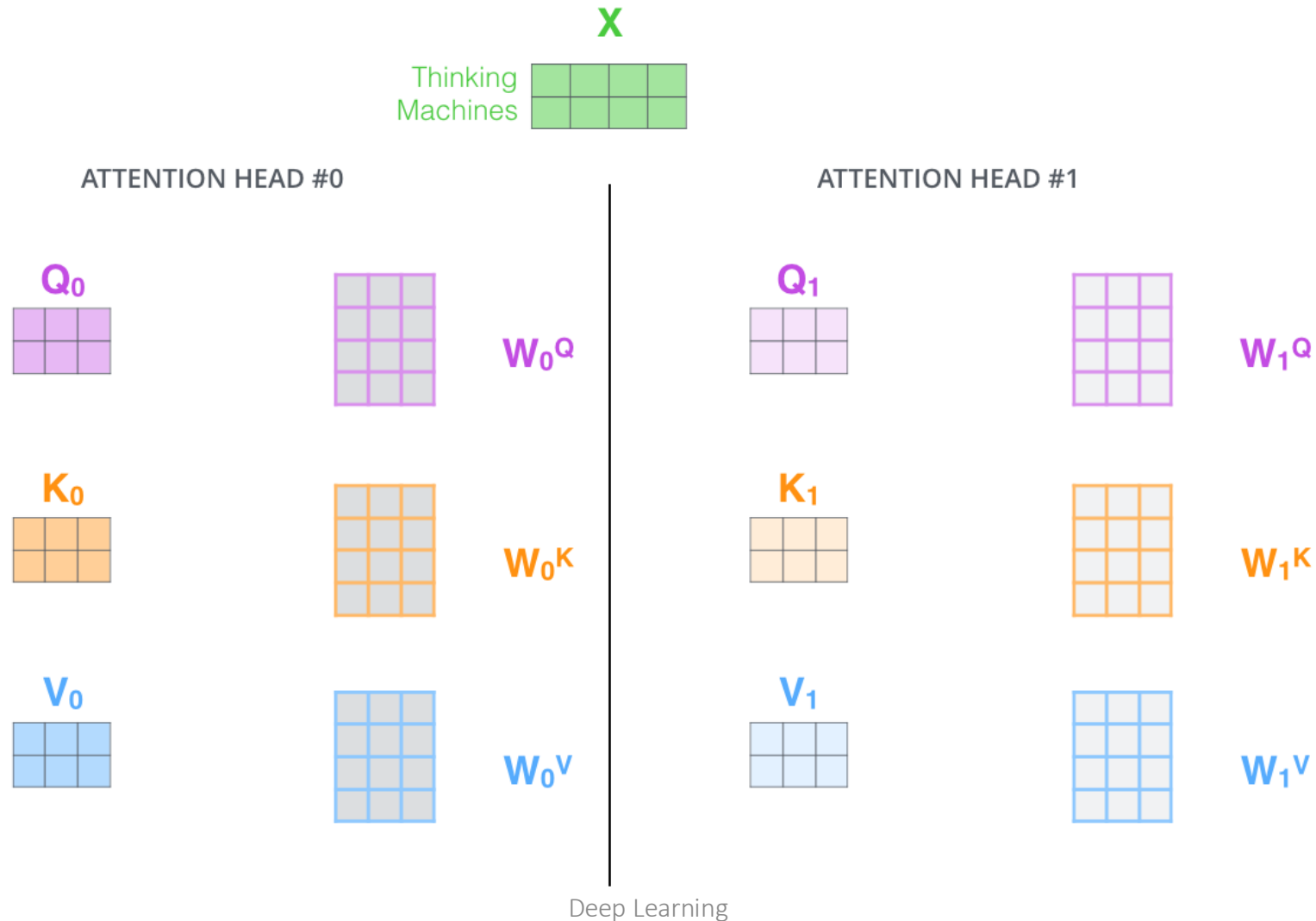
Self-Attention



Multi-Head Attention

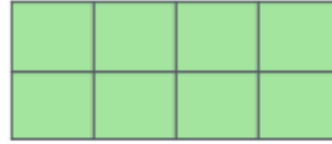


Multi-head attention



X

Thinking
Machines

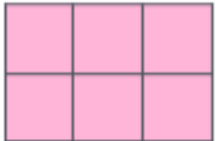


Calculating attention separately in
eight different attention heads



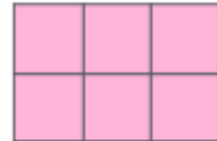
ATTENTION
HEAD #0

Z_0



ATTENTION
HEAD #1

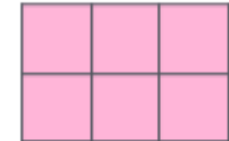
Z_1



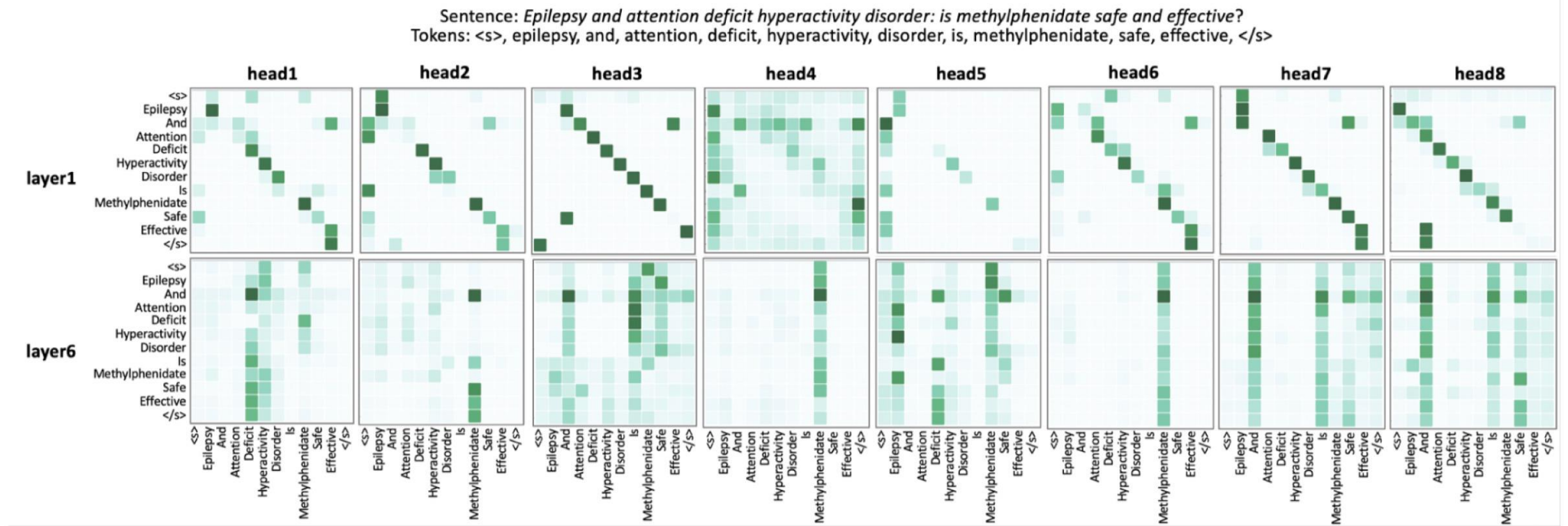
...

ATTENTION
HEAD #7

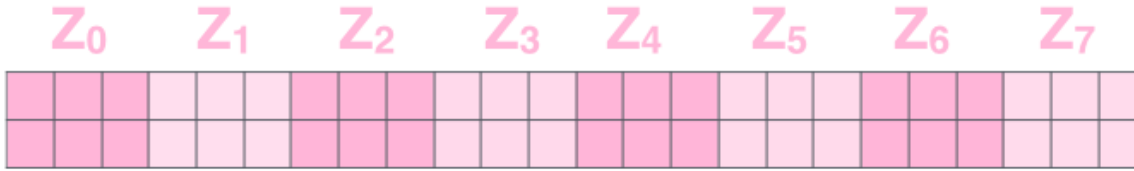
Z_7



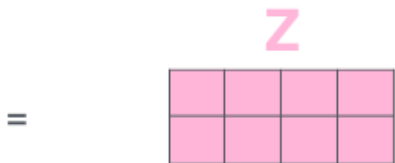
Multihead Visualization



1) Concatenate all the attention heads

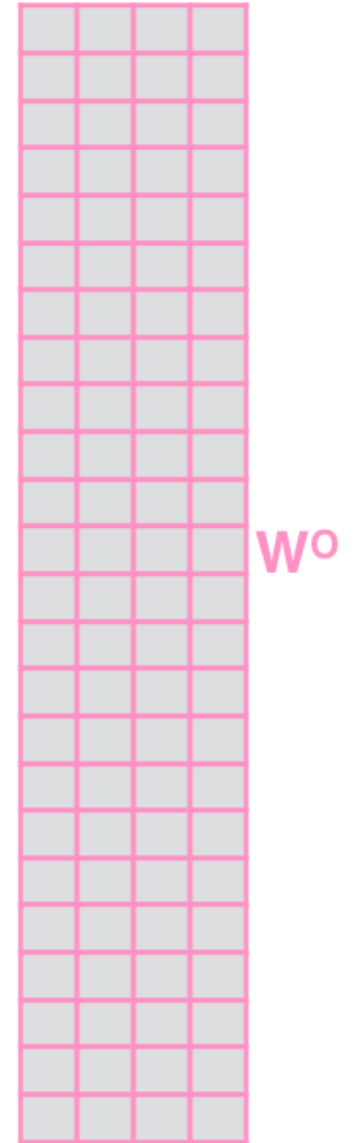


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



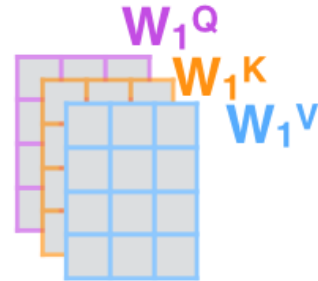
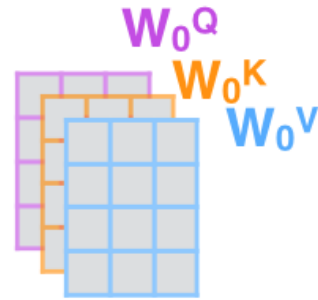
1) This is our input sentence*

Thinking
Machines

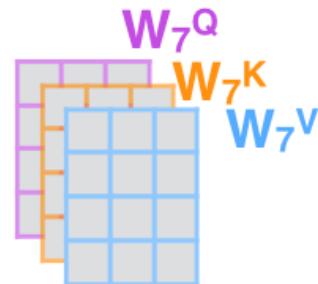
2) We embed each word*



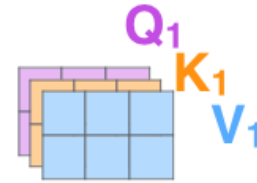
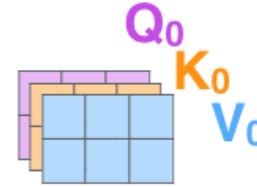
3) Split into 8 heads.
We multiply X or R with weight matrices



...



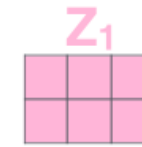
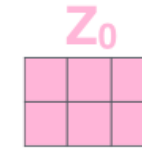
4) Calculate attention using the resulting $Q/K/V$ matrices



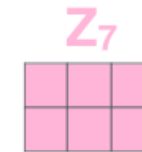
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



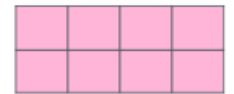
...



W^O

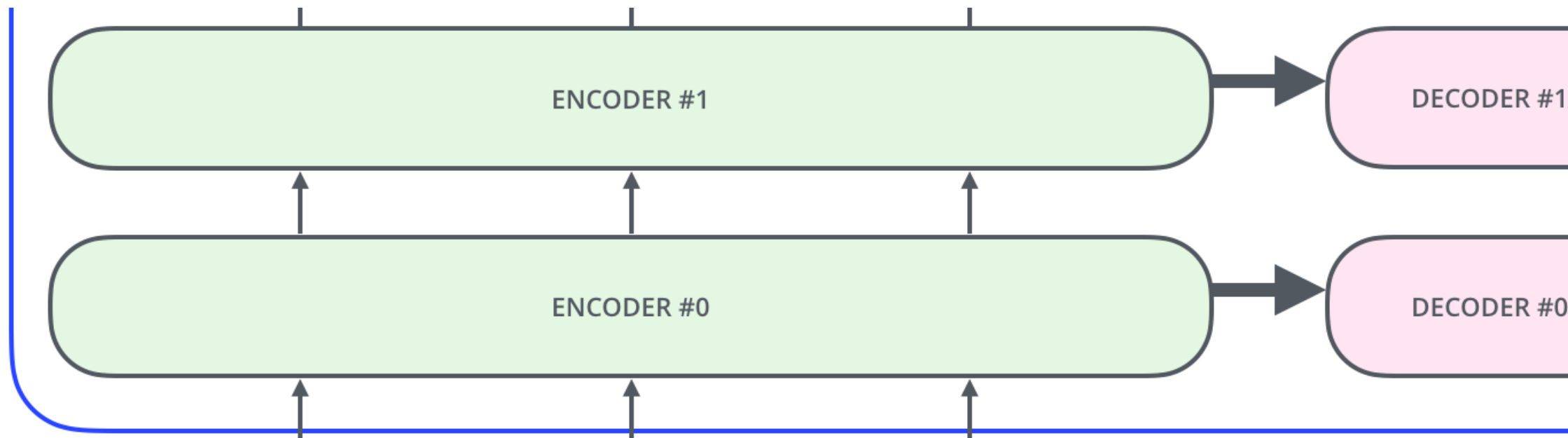


Z



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one





EMBEDDING
WITH TIME
SIGNAL

x_1

x_2

x_3

=

=

=

POSITIONAL
ENCODING

t_1

t_2

t_3

+

+

+

EMBEDDINGS

x_1

x_2

x_3

INPUT

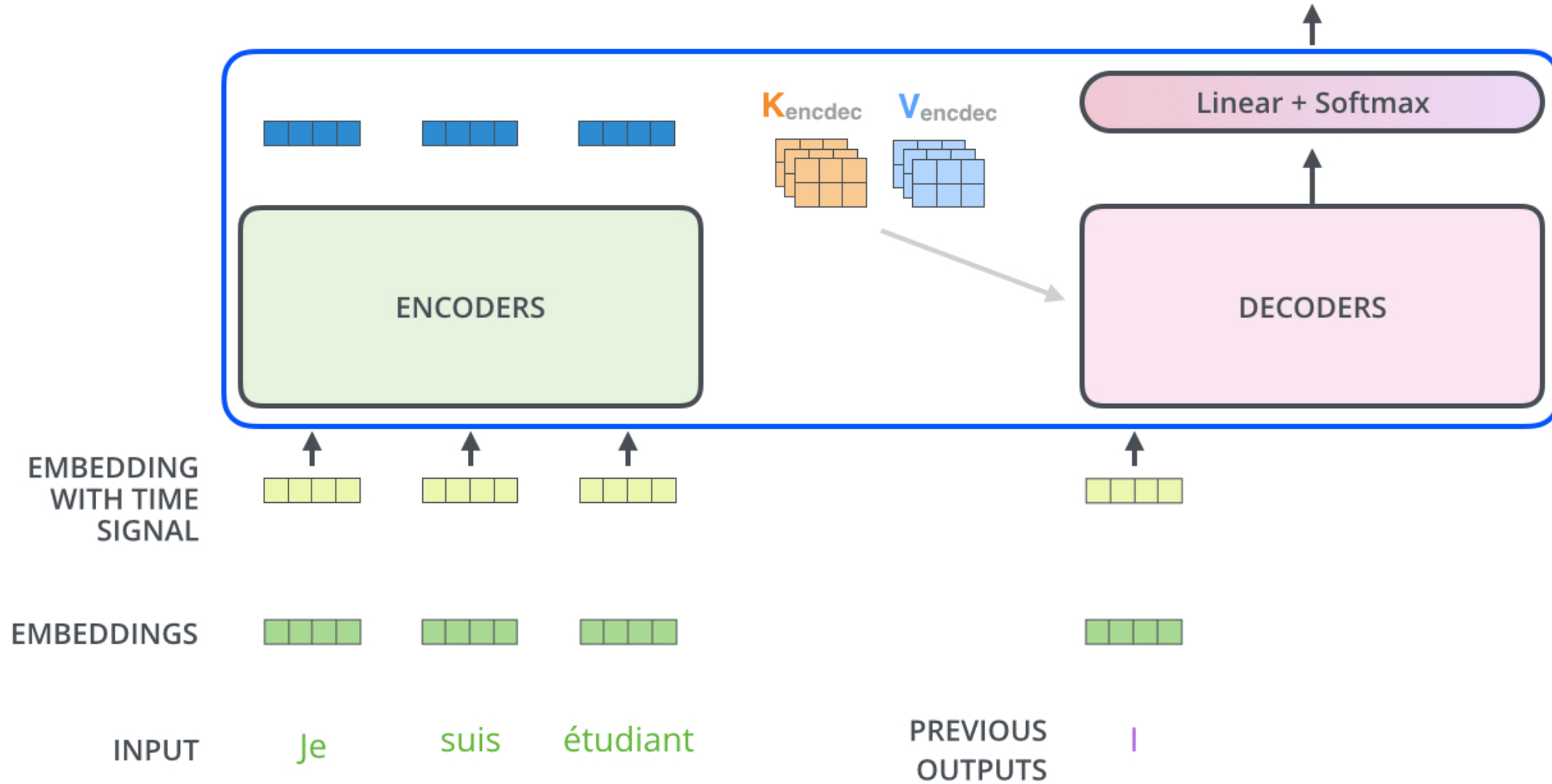
Je

suis

étudiant

Decoding time step: 1 2 3 4 5 6

OUTPUT |



Pre-training and Fine-tuning

Pre-training and Finetuning Paradigm

- **Language modeling** = learning to assign probabilities to sequences of words
 - Learns which sequences of words are most likely
- **Pretrained** language models can extract generalizations from large amounts of text

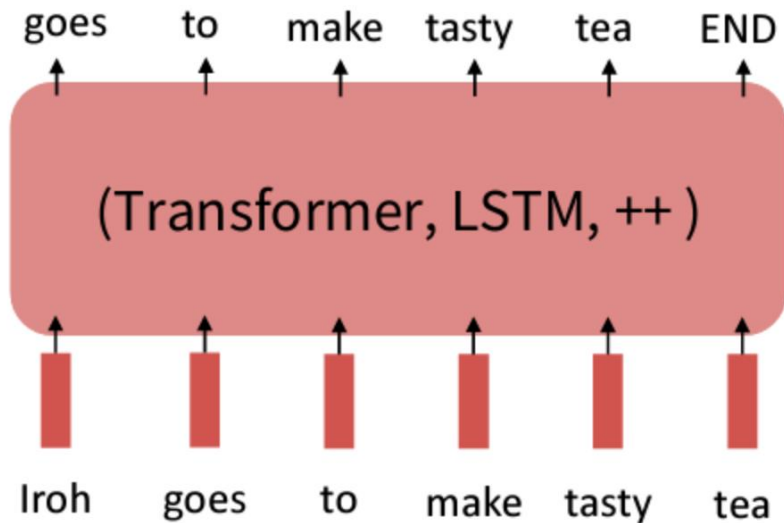
Pre-training and Finetuning Paradigm

- If we want to use a pretrained LM for a specific task, we need to **fine-tune** it!
 - Process of **taking the network learned by these pre-trained models**, and **further training the model**, often via an added neural net classifier that takes the top layer of the network as input, to perform some downstream task
 - Fine-tuning is a training process and takes gradient descent steps!

Pre-training and Finetuning Paradigm

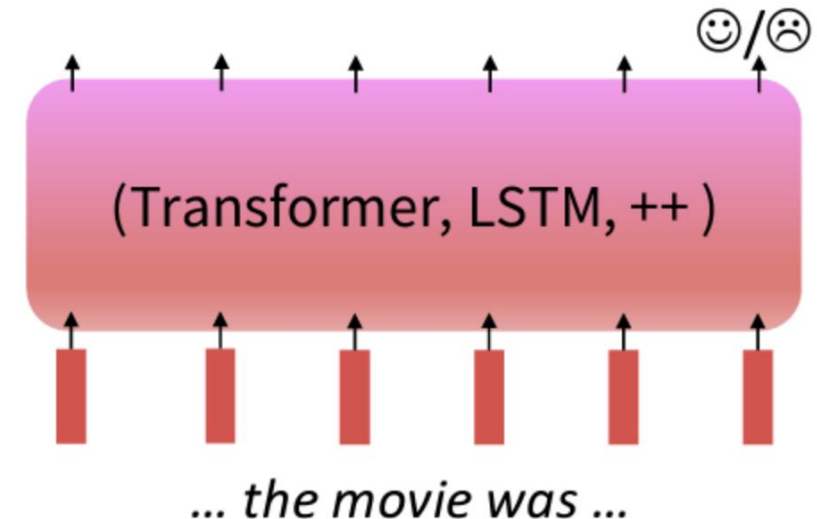
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

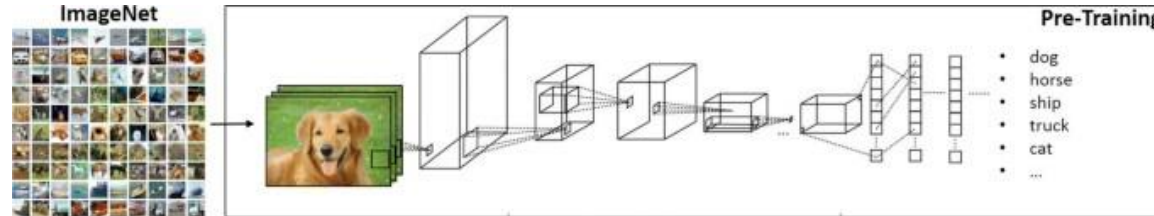
Not many labels; adapt to the task!



Slide Credit: Chris Manning, CS 224N

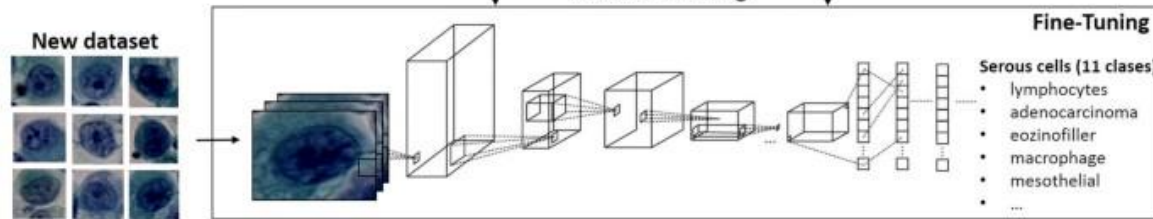
Pretraining / Fine-tuning Examples

Pre-training



1.28M images, 1000 classes

Fine-tuning



3652 images, 11 classes

Pre-training

Natural language [MASK] (NLP) is an [MASK] subfield of linguistics, computer science, and artificial [MASK] concerned with the interactions [MASK] computers and human [MASK] ...

processing,
interdisciplinary,
Intelligence,
between,
language

3.3B tokens
(512 tokens per segment)

Fine-tuning

contains no wit , only labored gags
the greatest musicians
very good viewing alternative

negative
positive
positive

67k examples, 2 classes

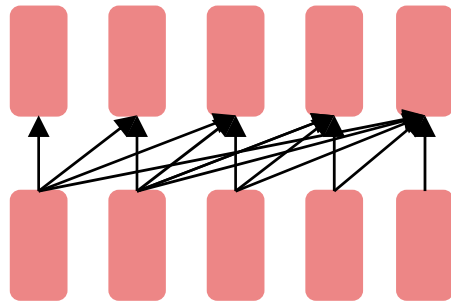
Large Language Models (LLMs)

Large Language Models (LLMs)

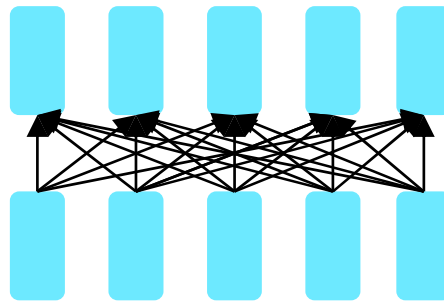
- Even though pretrained only to predict words, they earn a lot of useful language knowledge since training on a lot of text
- They are large models!
 - Components:
 - L = # of layers
 - H = hidden size (size of q , k , v vectors)
 - A = # of attention heads
 - Examples:
 - BERT Large: $L=24$, $H=1024$, $A=16$ (340 million parameters)
 - Llama 3: $L=126$, $H=16384$, $A=128$ (405 billion parameters)

↑ These become the size (dimensionality) of the representation!

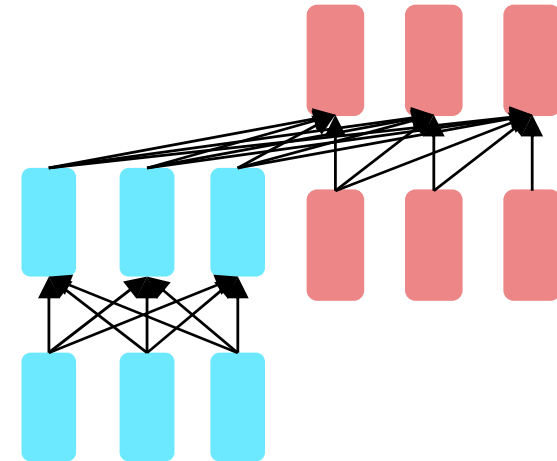
Three architectures for LLMs



Decoders
GPT, Claude,
Llama, Mistral

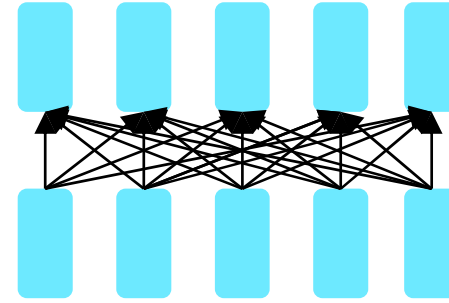


Encoders
BERT family,
HuBERT



Encoder-decoders
Flan-5, Whisper

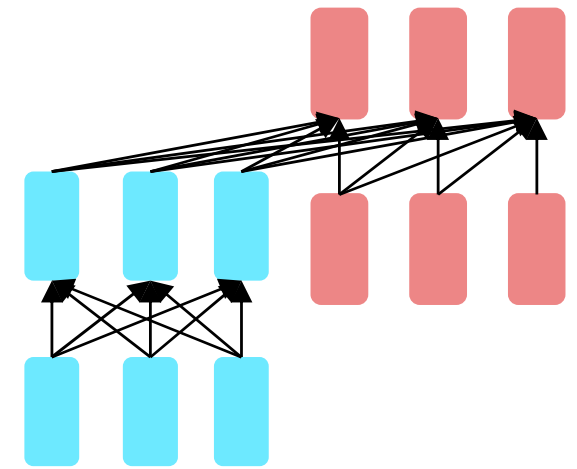
Encoders



- Many varieties!
- Popular: Masked Language Models (MLMs)
- BERT family
- Trained by predicting words from surrounding words on both sides
- Are usually **finetuned** (trained on supervised data) for classification tasks.

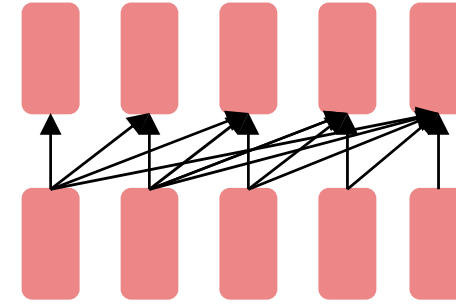
Encoder-Decoders

- Trained to map from one sequence to another
- Very popular for:
 - machine translation (map from one language to another)
 - speech recognition (map from acoustics to words)

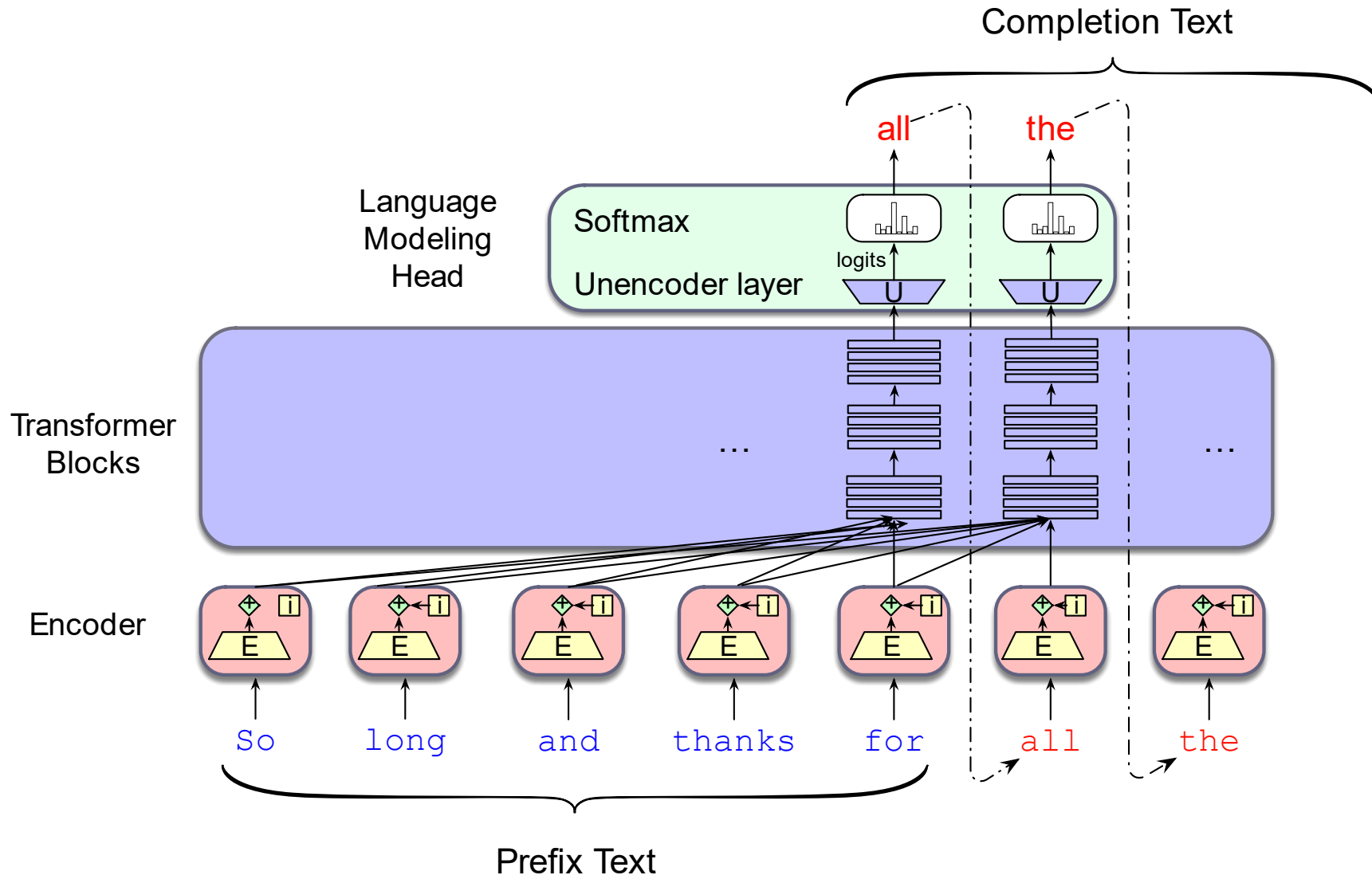


Decoder-only models

- Also called:
 - Causal LLMs
 - Autoregressive LLMs
 - Left-to-right LLMs
- Predict words left to right



Conditional Generation: Generating text conditioned on previous text!



Many practical NLP tasks can be cast as word prediction!

- Some common natural language processing (NLP) tasks



<https://www.qtravel.ai/blog/what-is-natural-language-processing-and-how-do-chatgpt-systems-work/>

Sentiment analysis

- “I like Jackie Chan”

1. We give the language model this string:

The sentiment of the sentence "I like Jackie Chan" is:

2. And see what word it thinks comes next:

$P(\text{positive} | \text{The sentiment of the sentence ``I like Jackie Chan" is:})$

$P(\text{negative} | \text{The sentiment of the sentence ``I like Jackie Chan" is:})$

Answering questions

QA: “Who wrote The Origin of Species”

- We give the language model this string:

Q: Who wrote the book ``The Origin of Species"? A:

- And see what word it thinks comes next:

$P(w|Q: \text{Who wrote the book ``The Origin of Species"? A:})$

- And iterate:

$P(w|Q: \text{Who wrote the book ``The Origin of Species"? A: Charles})$

Summarization

Original The only thing crazier than a guy in snowbound Massachusetts boxing up the powdery white stuff and offering it for sale online? People are actually buying it. For \$89, self-styled entrepreneur Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says.

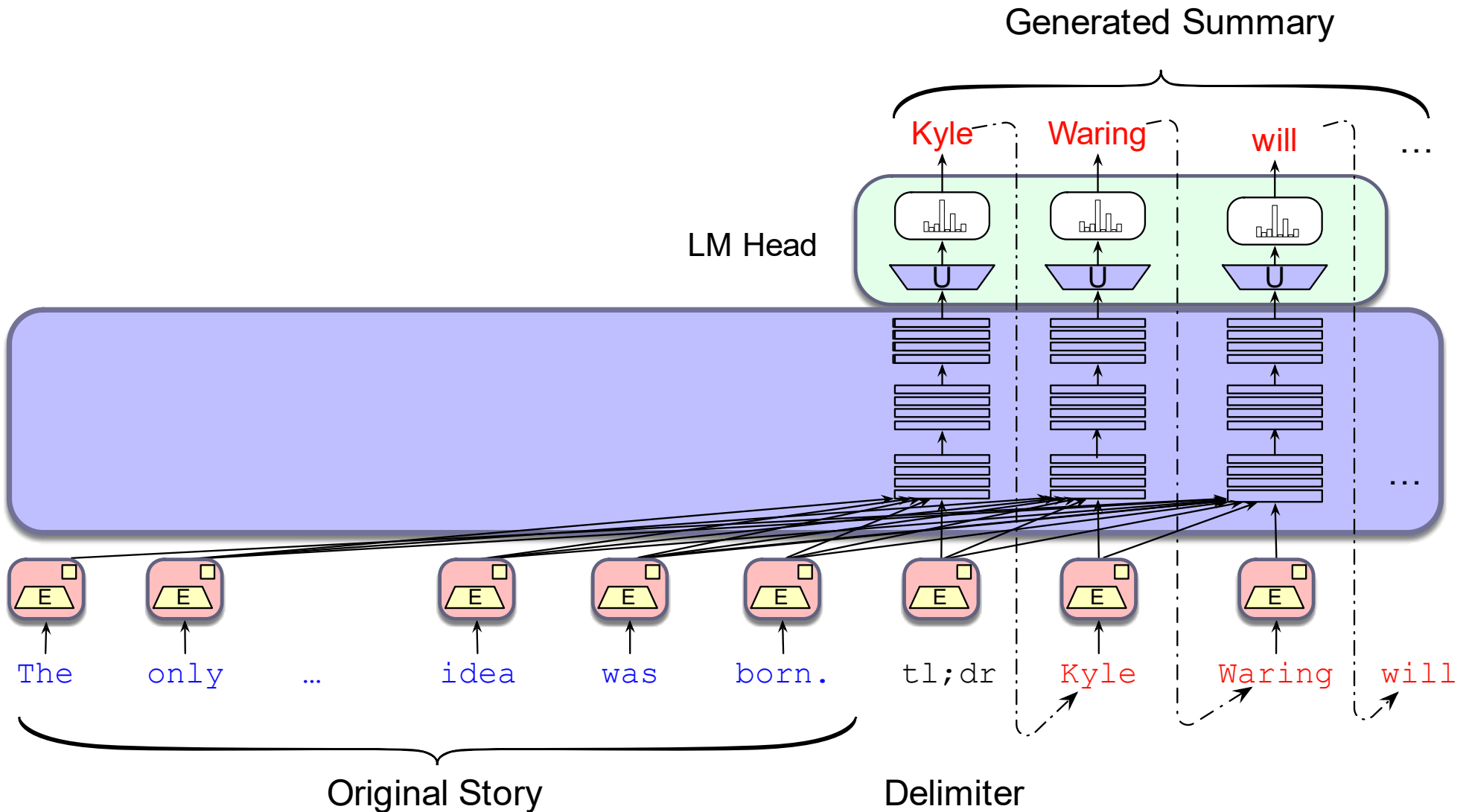
But not if you live in New England or surrounding states. “We will not ship snow to any states in the northeast!” says Waring’s website, ShipSnowYo.com. “We’re in the business of expunging snow!”

His website and social media accounts claim to have filled more than 133 orders for snow – more than 30 on Tuesday alone, his busiest day yet. With more than 45 total inches, Boston has set a record this winter for the snowiest month in its history. Most residents see the huge piles of snow choking their yards and sidewalks as a nuisance, but Waring saw an opportunity.

Summary According to Boston.com, it all started a few weeks ago, when Waring and his wife were shoveling deep snow from their yard in Manchester-by-the-Sea, a coastal suburb north of Boston. He joked about shipping the stuff to friends and family in warmer states, and an idea was born. [...]

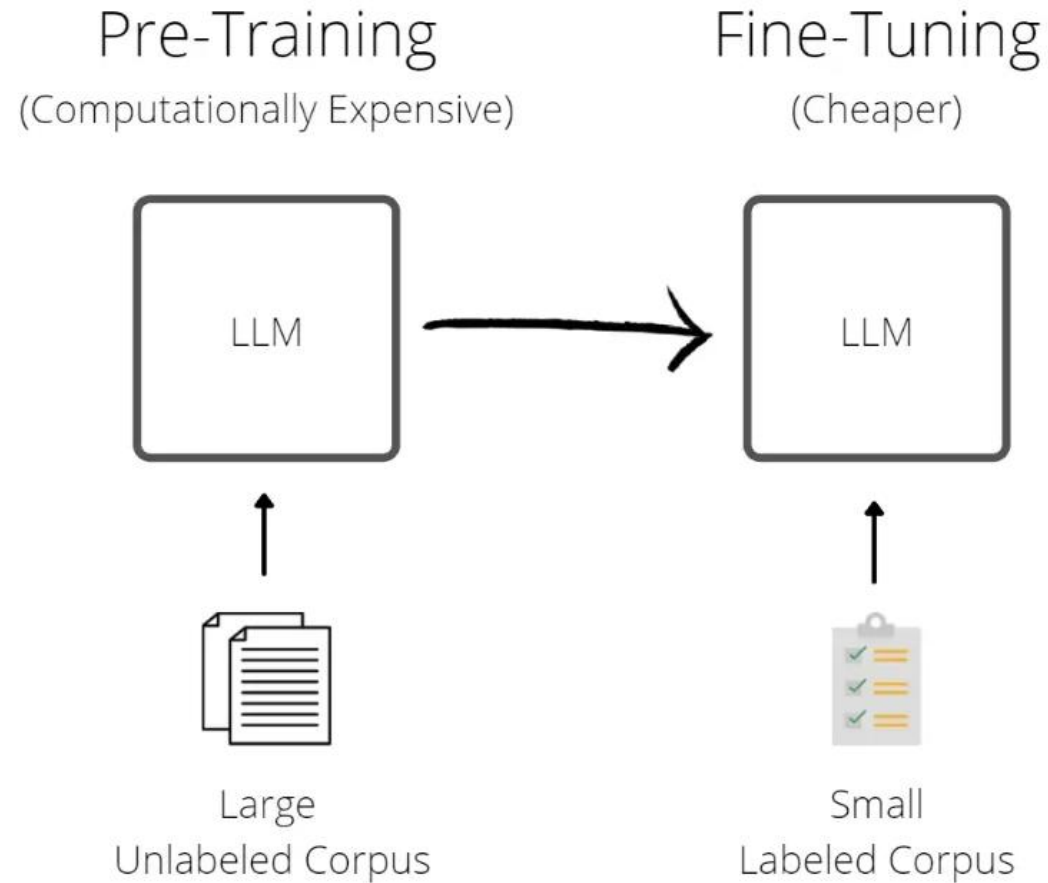
Kyle Waring will ship you 6 pounds of Boston-area snow in an insulated Styrofoam box – enough for 10 to 15 snowballs, he says. But not if you live in New England or surrounding states.

LLMs for summarization (using tl;dr)



How to train them?

- Two main stages:



<https://research.aimultiple.com/large-language-models/>

Pretraining

- First **pretrain** a transformer model on enormous amounts of text
 - Download and preprocess internet data. E.g., [fine web pipeline](#)
 - Tokenization: convert raw text into sequences of symbols (/tokens)
 - Then, self-supervised training

Tiktokenizer

gpt-4o

System

EECS 492 at UM!

×

User

Content

×

Add message

```
<|im_start|>system<|im_sep|>EECS 492 at UM!<|im_end|>
<|im_start|>user<|im_sep|><|im_end|>
<|im_start|>assistant<|im_sep|>
```

Token count

18

```
<|im_start|>system<|im_sep|>EECS 492 at UM!<|im_end|><
|im_start|>user<|im_sep|><|im_end|><|im_start|>assista
nt<|im_sep|>
```

```
200264, 17360, 200266, 10264, 9310, 220, 40719, 540, 5
1338, 0, 200265, 200264, 1428, 200266, 200265, 200264,
173781, 200266
```

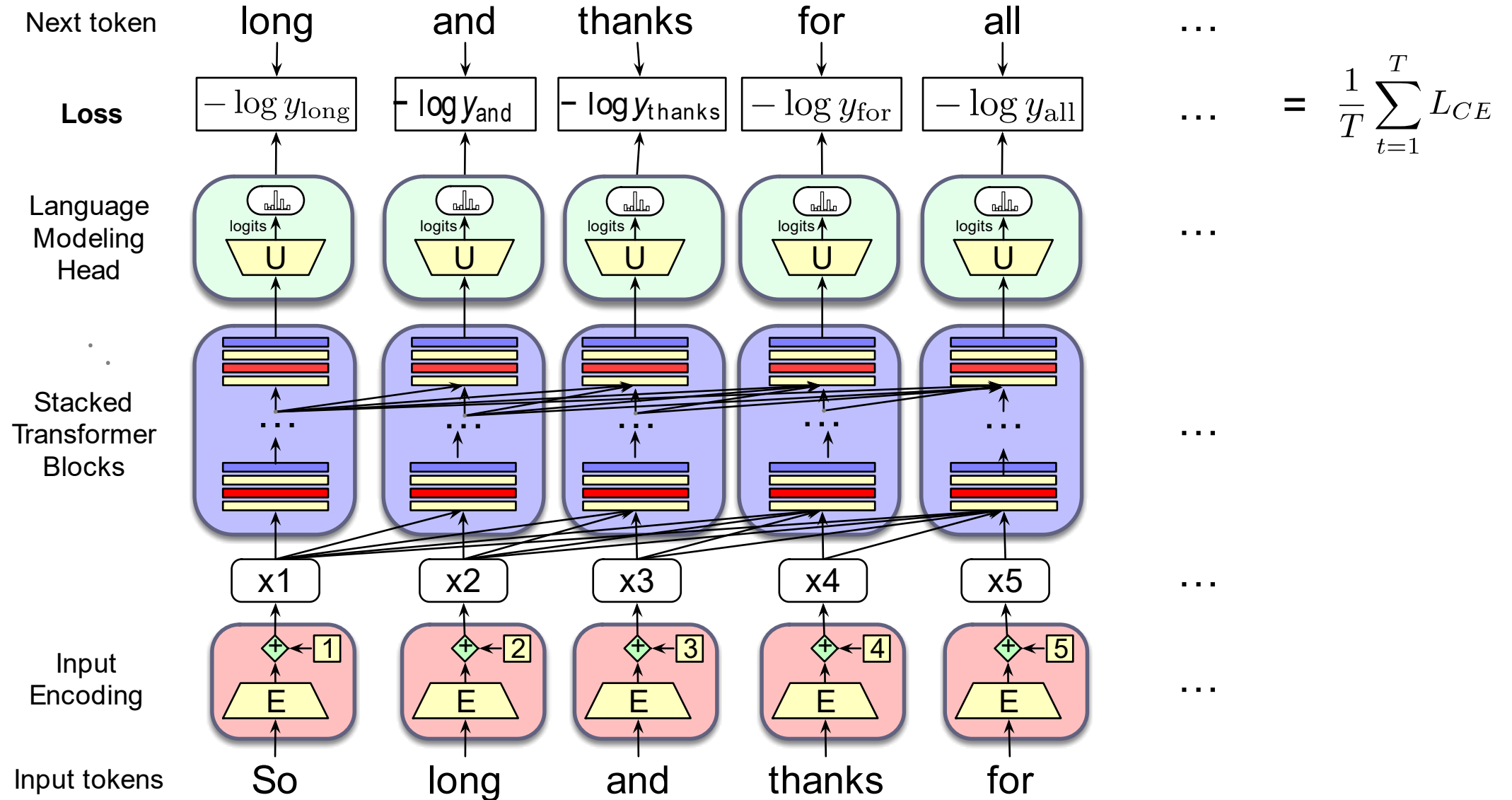
Deep Learning

<https://tiktokenizer.vercel.app/>

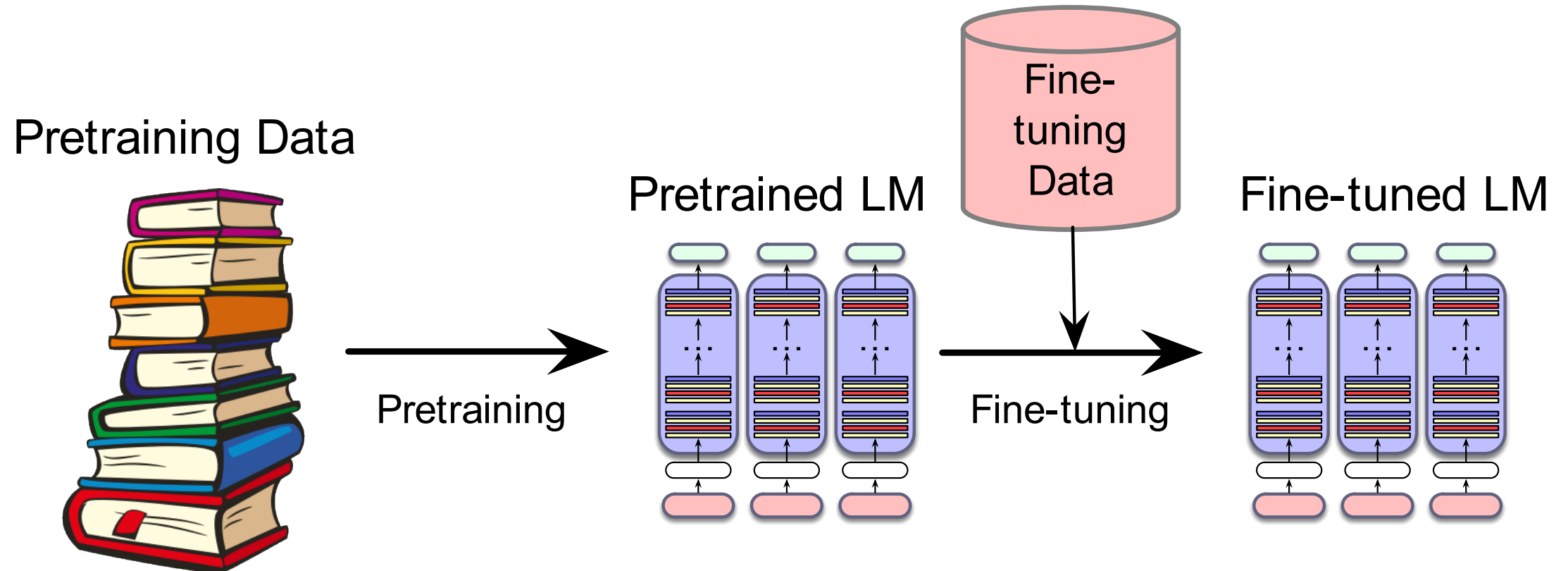
Self-supervised training algorithm

- We just train them to predict the next word!
 1. Take a corpus of text
 2. At each time step t
 - ask the model to predict the next word
 - train the model using gradient descent to minimize the error in this prediction
- "**Self-supervised**" because it just uses the next word as the label!
- What loss function? Cross-entropy (CE) loss

Training a transformer language model



Finetuning



Finetuning

- It means taking a pretrained model and further adapting some or all of its parameters to some new data
- There are many ways to do this. Let's look into one example.

Open AI's InstructGPT

RL w/ human feedback (RLHF)

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A B
C D
Explain gravity... Explain war...
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM
D > C > A = B

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO
Once upon a time...

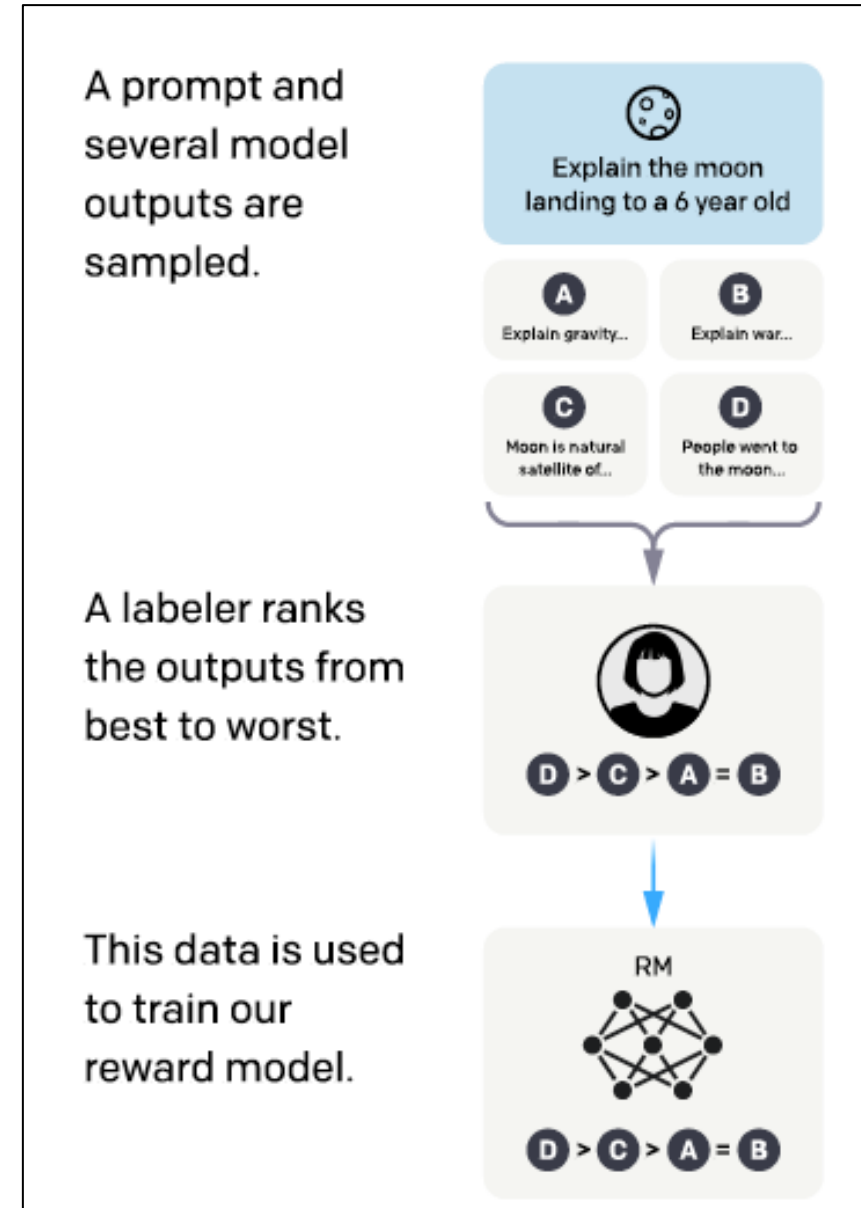
The reward model calculates a reward for the output.

RM
 r_k

The reward is used to update the policy using PPO.

Learning to reward LLMs

- Reward model r_θ is a neural network that is trained with labeler rankings.
- Labelers rank between $K = 4$ and $K = 9$ responses, and the reward model is trained on all $\binom{K}{2}$ comparisons:
$$\text{loss} = -\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))$$
 - x : prompt
 - y_w : preferred completion
 - y_l : less preferred completion



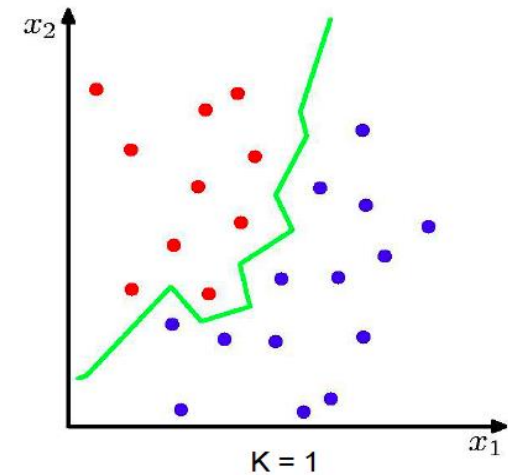
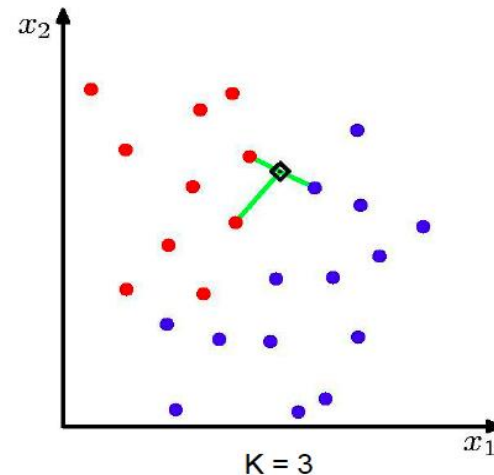
Deep Learning Summary

- Appealing due to brain analogy
- Advantages
 - Expressiveness
 - Many tools publicly available
 - Ability to handle noise
- Disadvantages
 - Opaque – cannot be used in some applications due to regulatory constraints
 - Art of designing structures and tuning parameters (especially true now...)

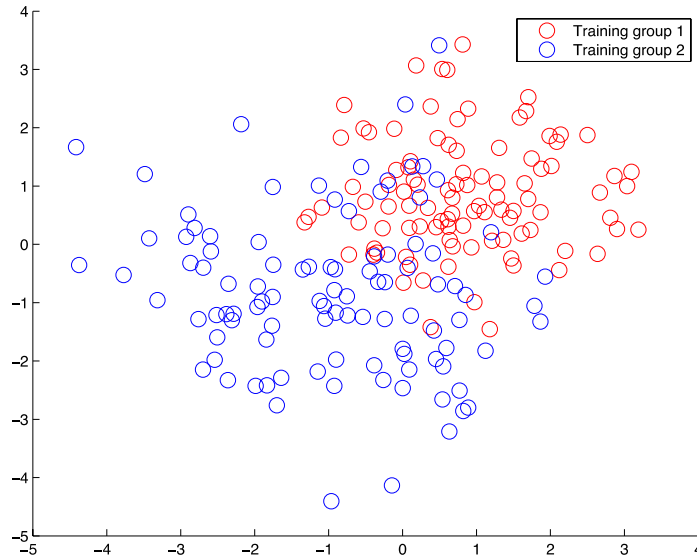
K- Nearest Neighbors

Nearest Neighbor Method

- What if we weren't concerned with an exact match?
- k -Nearest Neighbors
 - Given: query $\mathbf{x}_q$
 - Find: k examples nearest to $\mathbf{x}_q$
- Approach
 - Find the $NN(k, \mathbf{x}_q)$
 - Take a plurality vote of neighbors
 - Assign label to class with majority vote

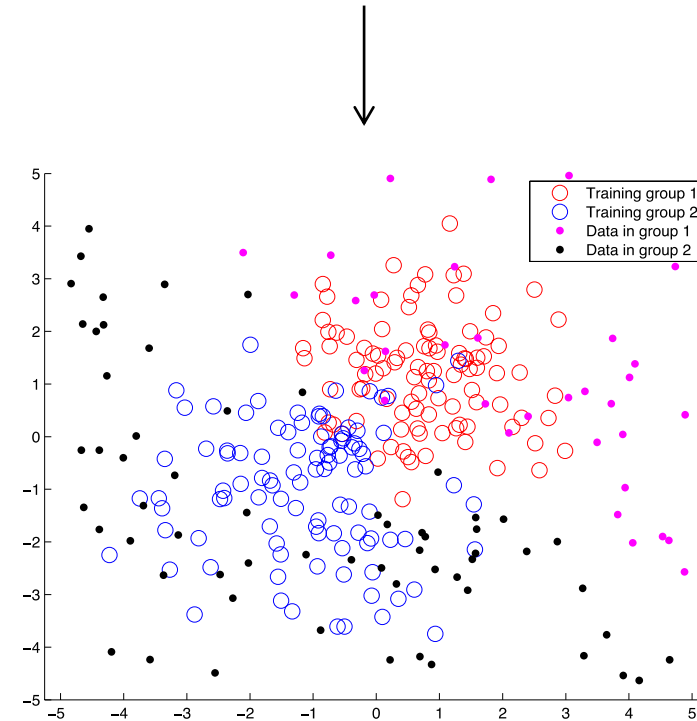


k -Nearest Neighbors – Example

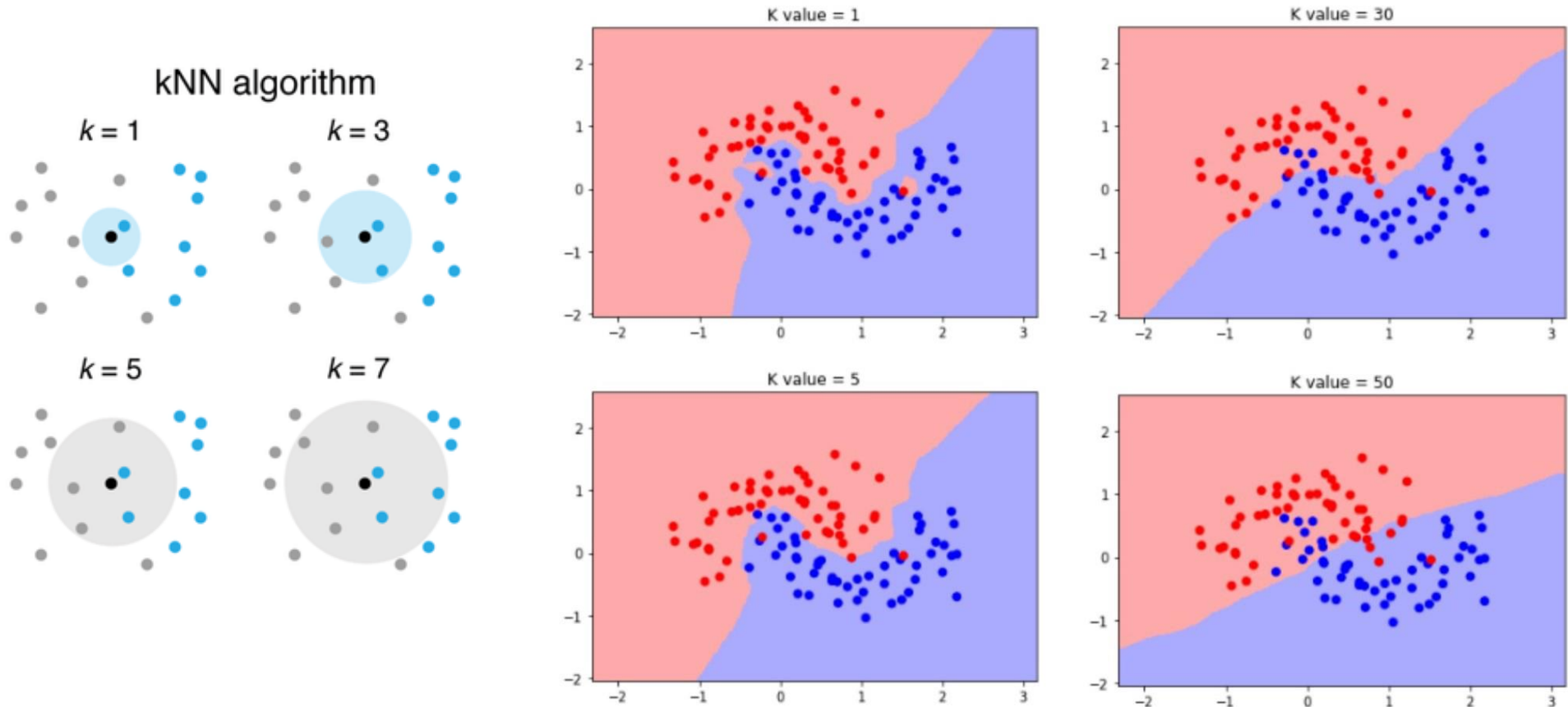


The original training data

Unlabeled testing data assigned based on k -Nearest Neighbors



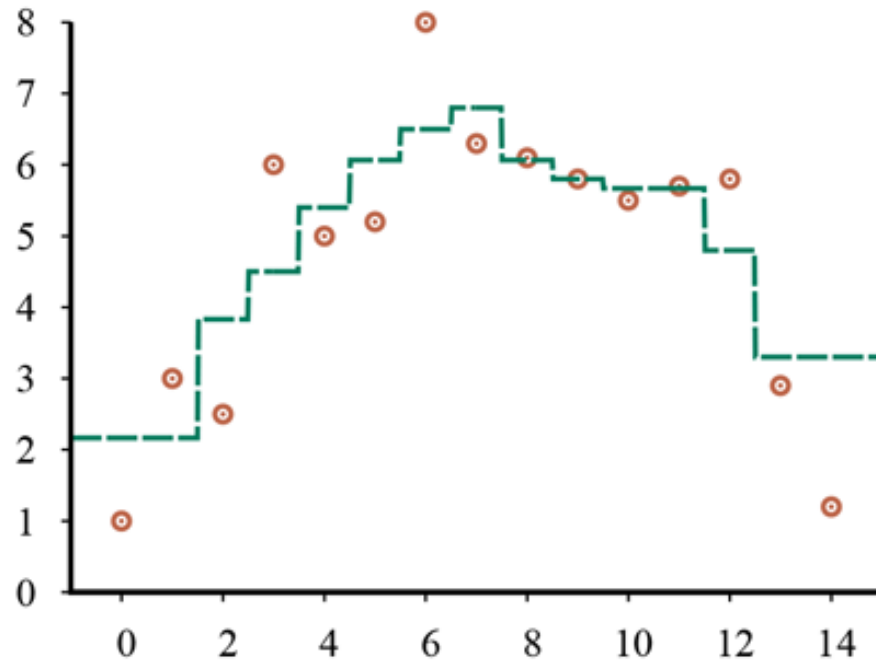
k -Nearest Neighbors – Effect of k



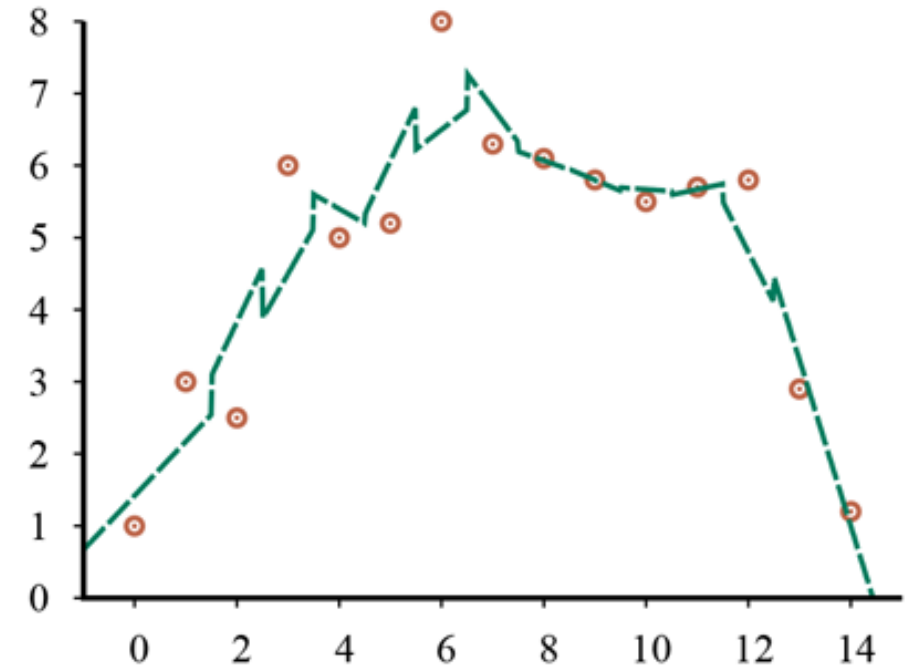
How should we calculate distance?

What about Regression?

3-nearest neighbors average



3-nearest-neighbors linear regression

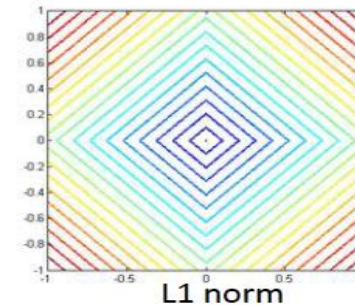


Distance Calculation in k -NN

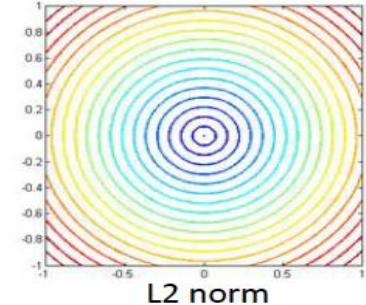
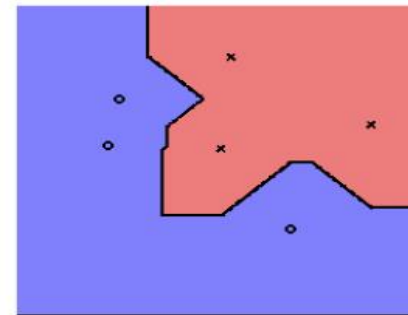
- Most common distance metric: Minkowski distance (L^p norm)

$$L^p(x_j, x_q) = \left(\sum_i |x_{j,i} - x_{q,i}|^p \right)^{1/p}$$

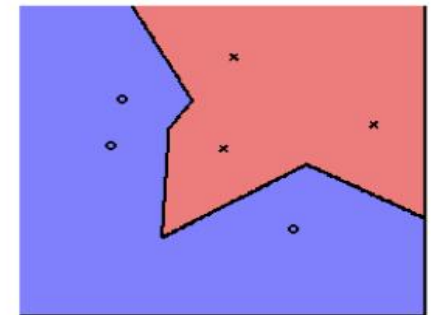
- When $p = 1$ – Manhattan distance
- When $p = 2$ – Euclidean distance



knn (K=1): L1 Distance



knn (K=1): L2 Distance



Normalization in k -NN

- Different features (dimensions) could have totally different units!
 - E.g., one feature is measured in miles, one measured in meters
 - Changing unit of a feature changes the distance calculation
- Common to apply [Z-normalization](#) in each dimension
 - Compute the mean μ_i and standard deviation σ_i of the values in each dimension i
 - Rescale each value: $x_{j,i}$ becomes $(x_{j,i} - \mu_i) / \sigma_i$

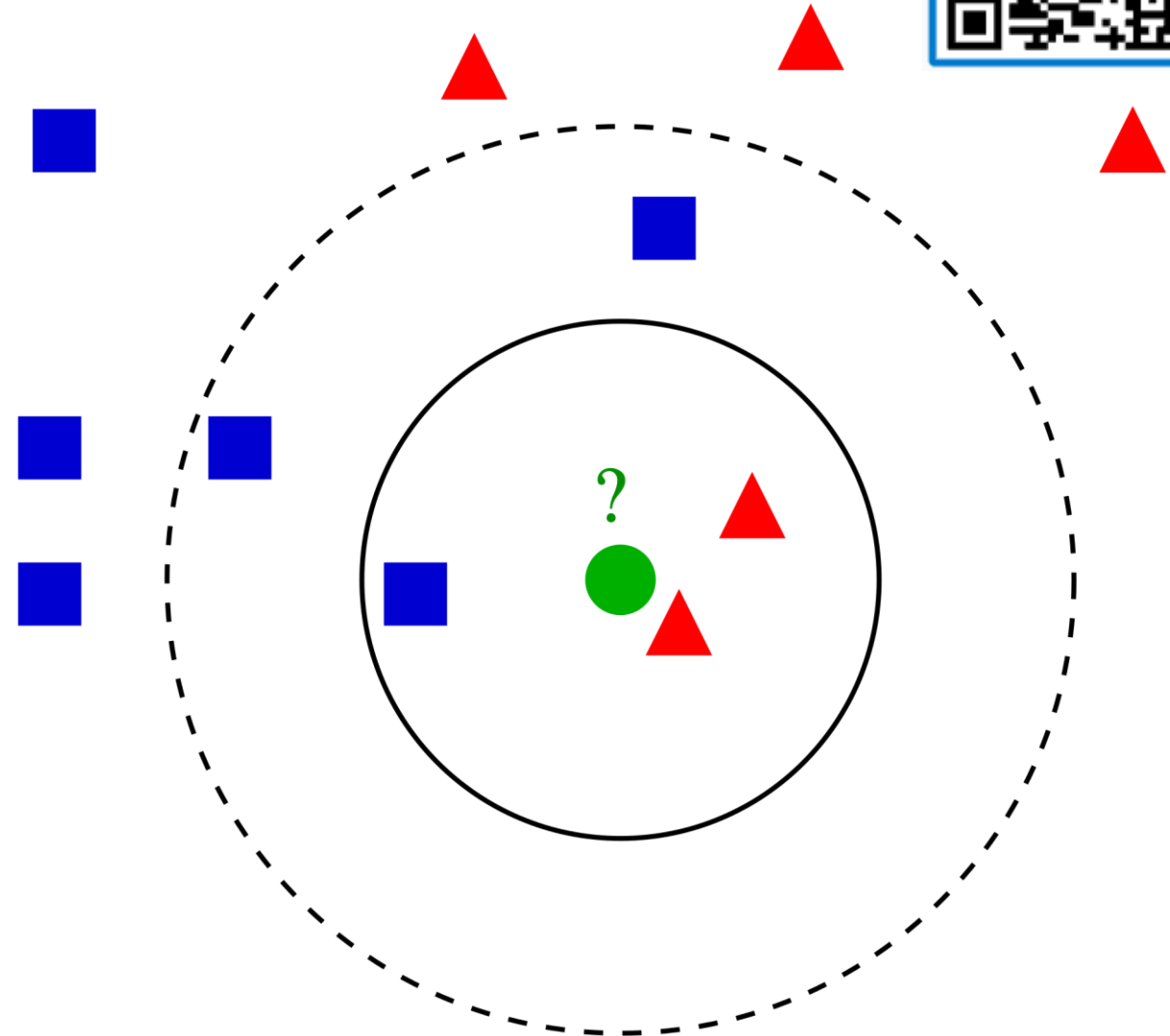
Curse of Dimensionality

- What happens in very high-dimensional spaces?
 - Points are farther away from each other!
- Let's say we have an n -dimensional unit hypercube
 - We have 1,000,000 points and we are looking for 10 nearest neighbors
 - How big of a neighborhood do we need to look in?
 - 2 dimensions $\rightarrow$ 0.003% of the edge length of the unit cube
 - 3 dimensions $\rightarrow$ 2% of the edge length of the unit cube
 - 17 dimensions $\rightarrow$ 50% of the edge length of the unit cube
 - 200 dimensions $\rightarrow$ 94% of the edge length of the unit cube



Exercise

- Will the new point (green dot) be classified as a blue square or a red triangle with 3 nearest neighbors?
- What about 5 nearest neighbors?
- How do we choose how many neighbors to consider?



Limitations

- What is another limitation of k -NN?
 - All calculations are done at run-time
 - How much training data must be stored?
- There are fixes to speed up k -NN
 - Example: k-d trees (section 19.7.2)
 - Example: Locality sensitive hashing (section 19.7.3)

Moving away from k -NN

- What is a limitation of k -NN?
 - All calculations are done at run-time
 - What does it mean for the utility of this classifier?
 - How much training data must be stored?
- There are fixes to speed up k -NN
 - Example: k-d trees (section 18.8.2)
 - Example: Locality sensitive hashing (section 18.8.3)
- What if we calculate our decision boundaries in advance?
 - Support Vector Machines!